Fused ensembles of dynamic-rupture earthquake simulations to accelerate Bayesian inference

Vikas Kurapati[1*], David Schneller[1], Linus Seelinger[2], Zihua Niu[3], Alice-Agnes Gabriel[4,3], Michael Bader[1]

[1*]School of Computation, Information and Technology, Technical University of Munich, Boltzmannstr. 3, Garching, 85748, Germany.
[2]Scientific Computing Center, Kalsruhe Institute of Technology, Kaiserstraße 12, Karlsruhe, 76131, Germany.
[3]Department of Earth and Environmental Sciences Geophysics, Ludwig-Maximilians-Universität, Theresienstr. 41, Munich, 80333, Germany.
[4]Scripps Institution of Oceanography, University of California, San Diego, 9500 Gilman Dr #0225, La Jolla, CA, United States of America.

*Corresponding author(s). E-mail(s): vikas.kurapati@tum.de;
Contributing authors: david.schneller@tum.de; linus.seelinger@kit.edu; zihua.niu@lmu.de; algabriel@ucsd.edu; bader@cit.tum.de;

Abstract

Understanding earthquake dynamics is essential for seismic hazard assessment and risk mitigation. In this context, Bayesian inference yields valuable insights into model parameters through a combination of simulation models and real-world data. Such Bayesian parameter inference with uncertainty quantification (UQ) requires numerous simulation runs and is therefore often computationally out of reach. Already a single high-fidelity earthquake simulation – governed by a linear hyperbolic seismic wave equation coupled nonlinearly to a friction law – is computationally expensive. In this study, we investigate the use of fused ensemble simulations as a means to accelerate large earthquake simulation workflows and UQ studies. Via fused ensembles, we turn the element-local small sparse/ dense matrix operations into tensor contractions working on a dense rank-3 tensor and sparse matrices. These are executed with better computational efficiency on CPUs, due to better exploitation of SIMD instructions. We implement and evaluate this approach in SeisSol, a high-performance computing software for simulation of complex earthquake events, which implements an Arbitrary high-order

DERivative Discontinuous Galerkin (ADER-DG) scheme. Our results demonstrate that fused simulations can be up to 4.56 times faster than with single execution – though depending strongly on discretization order, size of the problem and compute architecture. For a full UQ example workflow, we demonstrate savings of 18% of node hours for the entire workflow.

Keywords: uncertainty quantification, fused simulations, earthquake simulation, Bayesian inference, dynamic rupture, discontinuous Galerkin

1 Introduction

In computational earthquake science, many approaches rely on performing large ensembles of earthquake simulations. This includes approaches to physics-based seismic hazard assessment (Callaghan et al. 2024; Cui et al. 2013; Mert et al. 2016, e.g.), uncertainty quantification and Bayesian inference of modeling parameters (Premus et al. 2022; Schliwa et al. 2024, e.g.) or even rapid-response simulations (Zuccolo et al. 2025). Such *ensemble simulations* share the need for performing a vast number of simulations with small variations in the input parameters (such as frictional parameters governing dynamic rupture or plastic deformation) or underlying physical conditions (subsurface stress state, source characteristics, e.g.). As models for the physical processes governing earthquake rupture and seismic wave propagation are getting more and more complex, the computational effort for accurately simulating earthquake phenomena in realistic three-dimensional Earth models is growing, as well. A single high-resolution earthquake simulation can easily require tens to hundreds, or even thousands of compute nodes of a supercomputer over several hours (e.g., Akcelik et al. 2003; Cui et al. 2010; Heinecke et al. 2014).

The high computational cost of running large ensembles provides motivation to exploit all avenues for reducing both the invested compute hours and the time to completion of such studies. Efficient parallelization of UQ algorithms and of the simulation runs typically aims at the latter: while the most cost-efficient (in terms of invested compute hours) way to execute a UQ workflow might be to execute all simulations sequentially and on the smallest-possible number of compute nodes – to not lose any performance due to lack of scalability – keeping the time to completion of such a workflow within acceptable limits, requires parallelization of both, accepting a certain imperfection in parallel efficiency. On the other hand, using highly sophisticated UQ algorithms that reduce the number of simulation runs as much as possible, or optimizing the time-to-solution of the individual simulation runs, will attempt to reduce the invested compute hours for the ensemble simulations.

In this work, we explore the potential of so-called *fused ensemble simulations* (Breuer et al. 2017; Parno et al. 2021; Uphoff and Bader 2020, e.g.) as an additional avenue to accelerate large-scale UQ workflows. We extend the earthquake simulation software SeisSol (www.seissol.org) to support the execution of several earthquake model evaluations with dynamic rupture and off-fault plasticity into one single *fused* simulation run. This is achieved by adding an additional dimension

to all degree-of-freedom tensors, thus improving the exploitation of vector operations and alignment to cache lines, avoiding zero-padding for non-aligned compute loads. We integrate fused earthquake simulations into general, potentially complex UQ workflows, using the 'Uncertainty Quantification and Model Bridge' (UM-Bridge, https://github.com/um-bridge) (Seelinger et al. 2025), which provides a flexible interface between UQ algorithms and simulators. Specifically, we introduce a reusable component that combines UQ model evaluation requests in batches for fused execution.

We evaluate the performance gains due to fused simulations for a dynamic rupture community benchmark (Harris et al. 2018) problem (TPV13[1]) and for a typical production scenario – the 2023 Kahramanmaraş, Turkey, M_w 7.8–7.7 Earthquake doublet (Gabriel et al. 2023; Jia et al. 2023) – on the CPU-based supercomputers Frontera and SuperMUC-NG. Finally, we showcase the potential to increase the number of simulations performed per available compute resources for UQ workflows on a Bayesian inference scenario (Kruse et al. 2024; Niu 2025), to determine the posterior distribution of a depth-dependent rate-and-state friction parameter (Dunham et al. 2011) of the dynamic rupture process modeling the 2019 M_w 6.4 Searles Valley Earthquake.

In section 2, we outline the aspired workflows for Bayesian inference of parameters for large-scale physics-based earthquake simulations. We provide an overview of related work in section 3. In section 4 and section 5, we summarize the high-order discontinuous Galerkin discretization used in SeisSol, introduce our fusion strategy, and discuss the challenges of applying it to non-linear physics models. In section 6, we describe the software architecture to smoothly integrate fused simulations into *any* UQ workflow, by extending the UQ and Modeling Bridge *UM-Bridge* (Seelinger et al. 2025) by introducing a component that transparently batches simulation requests and forwards them to the model execution (with SeisSol). Section 7 is dedicated to performance and scaling results for the benchmark and production scenario, and section 8 presents results for exploiting fused simulations in an example workflow with UM-Bridge and SeisSol. Finally, we conclude the potential directions for future research in section 9.

2 Background: Bayesian inference of parameters for large-scale physics-based earthquake simulations

While fused ensembles can be considered for any simulation workflow that involves potentially parallel or concurrent execution of many simulation tasks (see section 3 for related work), our study focuses on a concrete setup: we consider *Bayesian inference* workflows to determine the probability distribution of physical parameters from observational data. In particular, we build on previous work (Kruse et al. 2024) on parallelization of the Multi-Level Delayed Acceptance (MLDA) algorithm (Lykkegaard et al. 2023), which examines the impact of three parallelization routes – parallel chains, additional prefetching (see overview below) and multi-node execution of the simulations themselves – on the overall performance of the UQ workflow. Respective workflows are designed for Bayesian inversion of parameters of complex 3D earthquake

[1] https://strike.scec.org/cvws/download/TPV12_13_Description_v6.pdf)

simulations, such as parallel work by Niu (2025), which resolves on- and off-fault parameters for dynamic rupture earthquake simulations with off-fault plasticity, using high-resolution satellite imagery and global navigation-system data.

We use SeisSol to simulate complex earthquake dynamics. SeisSol offers the following modeling features:

- seismic wave propagation in elastic, elastic-acoustic (Dumbser et al. 2007; Krenz et al. 2021) , visco-elastic (Käser et al. 2007; Uphoff and Bader 2016) and poro-elastic (Wolf et al. 2022) materials;
- dynamic rupture earthquake sources (Pelties et al. 2012) based on various friction laws like linear slip weakening (Ida 1972; Palmer et al. 1973), or rate and state friction laws (Dieterich 1979; Ruina 1983);
- off-fault inelastic deformation (Wollherr et al. 2018) (when stresses exceed the limit for elastic response), implemented using a visco-elasto-plastic Drucker-Prager rheology (Andrews 2005);
- clustered local time stepping (LTS) to use different time-steps in the computational domain with the ADER-DG method (Breuer et al. 2016; Uphoff et al. 2017).

In addition, Niu et al. (2025), implemented seismic wave propagation in nonlinear rock models in SeisSol. SeisSol uses high-order discontinuous Galerkin discretization with arbitrary high-order derivative time-stepping (ADER-DG) (Dumbser et al. 2007) on unstructured adaptive tetrahedral meshes – see section 4.1 for details.

In our target UQ workflows (Kruse et al. 2024; Niu 2025), we aim to determine the physical parameters governing on-fault friction in the dynamic rupture process and off-fault inelastic deformation due to strong dynamic stress variations, and how these can be inferred from surface displacement data. We quantify the uncertainties and correlations among the physical parameters via sampling-based Bayesian inversion approaches. In (Kruse et al. 2024), we introduced prefetching simulation runs as a way of parallelization in Multilevel Delayed Acceptance (MLDA) (Lykkegaard et al. 2023). MLDA is closely related to the earlier Multilevel MCMC approach of Dodwell et al. (2015), and to delayed acceptance MCMC (Christen and Fox 2005). Like its spiritual ancestor Multilevel MCMC (Dodwell et al. 2015), MLDA exploits a hierarchy of models with increasing fidelity and respective cost. Combining a few evaluations of the compute-intensive fully-resolved model with many evaluations of fast approximate models, it can significantly reduce compute cost while achieving accurate inversion results. In (Kruse et al. 2024), different convergence orders in ADER-DG (3 vs. 4) are chosen as levels of model accuracy, and approximately twice as many models of convergence order 3 are evaluated as at the finest level of convergence order 4.

We employ the UM-Bridge interface (Seelinger et al. 2025) in order to seamlessly integrate our UQ software with SeisSol running on a high-performance computing (HPC) cluster. In addition to acting as a language-agnostic bridge between simulators and higher-level analysis tools, UM-Bridge handles parallel simulation jobs on HPC systems in a way that is entirely transparent to UQ software. Thanks to this universal interface, any higher-level analysis workflow (including UQ, ML, and optimization) can now benefit from batched simulations, if these are supported by the forward simulator.

Assumptions and Potential:

In the following, we will combine potentially concurrent simulations in the workflow into *fused ensembles*. To efficiently implement such ensembles, we assume that all fused simulations work on a "similar" setup – more concretely, we demand that:

1. The discretized mesh is identical for all simulations, and all simulations have the same order of convergence. This ensures that all element updates (incl. neighbor contribution, etc.) follow the same numerical scheme for all fused simulations.
2. While this enforces a uniform fault geometry for all fused simulations, fault properties such as parameters for static or dynamic friction, etc., can be varied.
3. All the simulations share the same material parameters. This implies that all simulations can run at the same (local) time step sizes, which are constrained by material parameters and element sizes.
4. The simulated time of all fused simulations, as well as any output settings (frequency of output, position of receivers, etc.), is identical.

We expect that these limitations are typically fulfilled by many simulation workflows for various applications, particularly to model uncertainties, estimate parameter distributions, or computing inverse problems, where algorithms such as Monte-Carlo-type sampling and Bayesian inference require numerous simulation runs that do not change the simulation setup significantly.

To the best of our knowledge, the explicit strategy of fusing multiple simulations into a single execution of a complex, high-fidelity solver such as SeisSol has not been extensively coupled with UQ frameworks. Our work fills this gap by introducing and analyzing fused ensembles to improve throughput and scalability in large-scale Bayesian inference tasks involving earthquake simulations.

3 Related work

Fused simulations:

Breuer et al. (2017) investigated the possibility of fusing multiple simulations for seismic wave propagation, using the same ADER-DG method as SeisSol with an isotropic elastic model, reporting a 2.1× speedup for a fourth-order scheme when fusing eight simulations in a single run compared to eight individual simulations. We implement a similar approach for full earthquake simulation in SeisSol, exploiting its code generation approach based on the YATeTo package (Uphoff and Bader 2020) for small tensor operations.

Fused solution of problems is more commonly utilized in the context of solving large (typically sparse) systems of equations with multiple right-hand sides. For example, early work by Feng et al. (1995) introduced a block conjugate gradient method to solve symmetric and positive definite linear systems simultaneously with multiple right-hand sides, addressing requirements of non-linear structural analysis problems, where multiple right-hand sides correspond to multiple loading scenarios. Liu et al. (2012) presented an algorithmic approach to redesign sparse matrix-vector products within iterative solvers, introducing a generalized sparse-matrix-vector (SPMV) kernel

that computes the matrix product with multiple vectors simultaneously. Many respective approaches are oriented towards specific applications and/or specific compute architectures. Walden et al. (2016) uses fused multiple right-hand sides for a specific kernel in lattice quantum chromodynamics and optimizes for the Intel Knights Corner manycore architecture. Imamura et al. (2016) study performance of iterative solvers for multiple right-hand-side vectors for exascale candidate architecture. Richtmann et al. (2016) processes multiple right-hand sides simultaneously to better amortize the expensive setup step of the DD-α algebraic multigrid method. Bastian et al. (2020) implements fused ensembles in the Finite Element framework DUNE, to enable solving linear systems with multiple right-hand sides for general elliptic and parabolic PDE solvers. Kühn et al. (2023) address SIMD vectorization on CPU architectures, introducing a specific SIMD data type in their SParse Linear System Solver library (Spliss) for solving locally varying linear systems with multiple right-hand sides.

In computational fluid dynamics, ensemble averaging is used to retrieve statistics of turbulence quantities, e.g. In that context, Makarashvili et al. (2017) analyzes the potential performance benefits of using ensemble simulations to calculate statistical quantities in large eddy simulations. Gunzburger et al. (2018) uses the same technique to accelerate fused ensembles with different viscosities. Krasnopolsky (2018) considers incompressible turbulent flow simulations (using direct numerical simulation, DNS) based on optimizing solvers for multiple right-hand sides. Mohebujjaman and Rebholz (2017) proposed an algorithm for computing flow ensembles of incompressible magneto-hydrodynamic flows under uncertainties in initial or boundary data by exploiting block linear solvers.

Phipps et al. (2017) envisage UQ workflows, in particular, realizing multiple-right-hand-side solvers for elliptical problems in the Trilinos package (Heroux et al. 2005), based on the performance portability framework Kokkos (Carter Edwards et al. 2014).

Earthquake simulation workflows:

Modern earthquake simulation frameworks integrate earthquake source characteristics, seismic wave propagation, and hazard analysis on HPC platforms. For example, the Statewide California Earthquake Center (SCEC)'s Cybershake system computes physics-based seismic hazard by simulating kinematic finite fault earthquake scenarios and 3D seismic wavefield simulations in a unified HPC workflow (Rojas et al. 2024; Callaghan et al. 2024). High-order codes like SeisSol and SW4 are leveraged to model seismic wave propagation in fully 3D heterogeneous domains on supercomputers(Taufiqurrahman et al. 2022; Petersson et al. 2023). These workflows employ automation to orchestrate thousands of parallel runs; for example, a recent CyberShake campaign executed around 32,000 jobs with 2.5 PB of output on the Summit supercomputer (Callaghan et al. 2024). Prototype rapid-response systems like UrgentShake also leverage HPC to generate physics-based ground motion scenarios in near real time (Zuccolo et al. 2025). Such integrated HPC-enabled workflows thus link detailed rupture modeling, full-wave propagation, and hazard outputs in a single workflow.

4 Physics-based earthquake simulations using SeisSol

In this section, we outline the numerical algorithms in SeisSol, as far as they are required to understand the scheme of fusing the simulations. We focus on the ADER-DG scheme for wave propagation (section 4.1), the dynamic rupture process (section 4.2), and treatment of off-fault plasticity(section 4.3).

4.1 The ADER-DG scheme for elastic wave propagation

We consider the elastic wave equation in first-order form (using Einstein convention)

$$\frac{\partial q_i}{\partial t} + A_{ij}\frac{\partial q_j}{\partial x} + B_{ij}\frac{\partial q_j}{\partial y} + C_{ij}\frac{\partial q_j}{\partial z} = 0, \tag{1}$$

where $\mathbf{q}$ contains the space-time dependent state variables

$$\mathbf{q} = (q_1, \ldots, q_9)^T := (\sigma_{11}, \sigma_{22}, \sigma_{33}, \sigma_{12}, \sigma_{23}, \sigma_{13}, v_1, v_2, v_3)^T, \tag{2}$$

in which σ_{ij} are the components of the symmetric stress tensor and v_1, v_2, v_3 are particle velocities in x, y, z directions. The space-dependent linear operators $\mathbf{A}, \mathbf{B}, \mathbf{C}$[2] govern the linear PDE system, and depend on the local material properties (Dumbser and Käser 2006). SeisSol solves the PDE system (1) on an unstructured tetrahedral mesh, using high-order discontinuous Galerkin discretization with Arbitrary high-order Derivatives (ADER, Titarev and Toro (2002); Dumbser et al. (2008)) time-stepping. The resulting ADER-DG scheme combines a high-order element-local predictor with a corrector step that considers how element-local solutions depend on neighbor elements by solving or approximating the Riemann problem at the element faces. We refer readers to (Käser and Dumbser 2006; Dumbser and Käser 2006; Dumbser et al. 2007) for details of the ADER-DG method and to (Uphoff et al. 2017; Uphoff and Bader 2020) for details of implementation in SeisSol.

For the context of this paper, we will focus on how the respective numerical steps are implemented in SeisSol. In each element k, the numerical solution of timestep n is stored as a matrix $\mathbf{Q}_k^n$ of polynomial coefficients with the quantities and polynomial basis functions as row and column dimensions of the matrix, respectively. Choosing orthogonal polynomial basis functions in space, DG discretization of eq. (1) yields the discretized derivative operator (omitting indices k and n for better readability)

$$D_{ij} := \hat{K}^{\xi}_{im} Q_{ml} A^*_{lj} + \hat{K}^{\eta}_{im} Q_{ml} B^*_{lj} + \hat{K}^{\zeta}_{im} Q_{ml} C^*_{lj}, \tag{3}$$

where $\hat{\mathbf{K}}^{\xi} = \mathbf{M}^{-1}\left(\mathbf{K}^{\xi}\right)^T$ (similar for $\hat{\mathbf{K}}^{\eta}, \hat{\mathbf{K}}^{\zeta}$), with stiffness matrices $\mathbf{K}^{\xi}$, $\mathbf{K}^{\eta}$ and $\mathbf{K}^{\zeta}$ and diagonal mass matrix $\mathbf{M}$. $\mathbf{A}^*$, $\mathbf{B}^*$ and $\mathbf{C}^*$ are linear combinations (depending on the geometric orientation of the element) of the matrices $\mathbf{A}$, $\mathbf{B}$ and $\mathbf{C}$ from (1).

[2] In the following, we denote vector, matrix, and tensor variables, such as $\mathbf{q}, \mathbf{A}$, etc. in bold font, and use q_i, A_{ij}, etc. for the respective vector, matrix, or tensor elements.

Following the Cauchy–Kovalevskaya procedure, the first step in the ADER-DG scheme is now to compute an element-local space-time predictor solution

$$\mathbf{L}(t+\tau) = \mathbf{Q}(t) + \tau \mathbf{D}^1 + ... + \frac{\tau^P}{P!}\mathbf{D}^P, \tag{4}$$

where P is the degree of the polynomial basic functions of the DG scheme, leading to a convergence order of $P+1$ (Cockburn et al. 2000). $\mathbf{D}^p$ is an approximation of the p-th derivative. The $\mathbf{D}^1, ..., \mathbf{D}^P$ can be computed by recursively applying eq. (3):

$$D_{ij}^{p+1} := \hat{K}_{im}^{\xi} D_{ml}^{p} A_{lj}^{*} + \hat{K}_{im}^{\eta} D_{ml}^{p} B_{lj}^{*} + \hat{K}_{im}^{\zeta} D_{ml}^{p} C_{lj}^{*} \qquad D_{ij}^{0} := Q_{ij}. \tag{5}$$

Integrating eq. (4) over a time step Δt yields the *predictor* values

$$I_{ij} = \sum_{p=0}^{P} \frac{\Delta t^{p+1}}{(p+1)!} D_{ij}^{p}, \tag{6}$$

from which we compute the updated unknowns $Q_{ij}^{n+1,*}$ for timestep $n+1$ via a *local corrector* step. The latter consists of a cell-local part,

$$\begin{aligned} Q_{ij}^{n+1,*} =& Q_{ij}^{n} + \tilde{K}_{im}^{\xi} I_{ml} A_{lj}^{*} + \tilde{K}_{im}^{\eta} I_{ml} B_{lj}^{*} + \tilde{K}_{im}^{\zeta} I_{ml} C_{lj}^{*} \\ & - \frac{1}{|J|} M_{il}^{-1} \left(\sum_{f=1}^{4} |S_f| \, F_{lm}^{-,f} I_{mn} A_{nj}^{*+} \right) \end{aligned} \tag{7}$$

and time-integrated contributions I_{mn}^{f} from the four neighbor elements $f = 1, \ldots, 4$, included via a *neighbor corrector*:

$$Q_{ij}^{n+1} = Q_{ij}^{n+1,*} - \frac{1}{|J|} \mathbf{M}_{il}^{-1} \left(\sum_{f=1}^{4} |S_f| \, F_{lm}^{+,i} I_{mn}^{f} A_{nj}^{*-} \right). \tag{8}$$

In eqs. (7) and (8), $\tilde{\mathbf{K}}^{\xi} = \mathbf{M}^{-1}\mathbf{K}^{\xi}$ (η, ζ similar), $|J|$ is the volume of the element, and $|S_i|$ the surface area of the i-th face. The flux matrices $\mathbf{F}^{-,i}$, $\mathbf{F}^{+,i,j,h}$ for each face i depend on the choice of the basis function and the relative position of the element with the respective neighbor (Dumbser and Käser 2006). $\mathbf{A}^{*+}_{k}$, $\mathbf{A}^{*-}_{k(i)}$ are the matrices considering the solution of the elastic Riemann problem at the boundaries which contribute to the numerical fluxes.

The entire predictor-corrector scheme, as outlined in eqs. (5) to (8), can be expressed as element-local matrix and/or tensor operations and is implemented via many small matrix multiplications (or tensor contractions), as explained in section 4.4.

4.2 Dynamic rupture

SeisSol simulates the dynamic rupture process by modeling fault slip as an internal boundary condition coupled to wave propagation (de la Puente et al. 2009; Pelties et al. 2012). First, the tractions (aligned to the fault plane, as resulting from the stresses) and velocities on both sides of the prescribed fault are computed. We then check if the fault remains locked or is sliding. Sliding causes a relative displacement across the fault interface, which is called slip. The behavior of sliding and slip is governed by **nonlinear friction laws**, which relate the shear traction to slip rate.

Traction $\mathbf{t}$ is split into shear traction $\boldsymbol{\tau}$ and a normal component $\mathbf{t}_\mathrm{n}$, with $\mathbf{t} = \boldsymbol{\tau} + \mathbf{t}_\mathrm{n}$. The fault slides, if the shear traction is larger than the fault strength

$$\tau_s = \max(0, -\mu_f t_\mathrm{n} - C). \tag{9}$$

C is a constant cohesion parameter which adds to the fault strength and $t_\mathrm{n} = \mathbf{t}_\mathrm{n} \cdot \mathbf{n}$ is the normal component of the traction. Note that, by convention, a negative t_n denotes compression. Following eq. (9), the fault strength τ_s is connected to the normal component t_n of the traction via a parameter μ_f, which is determined by a friction law

$$\mu_f = f(||\mathbf{s}||, \psi), \qquad \frac{\partial \psi}{\partial t} = g(||\mathbf{s}||, \psi), \tag{10}$$

where the friction law functions f and g dependent on a state variable ψ and on the slip rate $\mathbf{s} = [\![\mathbf{v}]\!]$, which is the difference in velocities across the fault. The differential equation (10) and eq. (9) need to be solved iteratively to obtain the tractions and velocities across the fault that satisfy the partial differential equation.

Thus, the dynamic rupture implementation in SeisSol performs the following steps:

1. We start from the element-local space-time predictor solution $\mathbf{L}(t + \tau)$ as defined in eq. (4) and evaluate the 3D space solution at $P + 1$ time quadrature points
2. For each time quadrature point, we project the 3D space solution onto the 2D dynamic rupture face, which gives the values of the stresses and velocities at each space-time quadrature point. This is done on both sides of the fault
3. At each space-time quadrature point on the face, we evaluate an intermediate state by solving the Riemann problem (assuming elasticity), using the values of stresses and velocities on either side of the fault.
4. With the obtained intermediate state, we check if the fault is locked or sliding by evaluating the fault strength as per eq. (9). If the fault is locked, we assume the intermediate state is the final state for flux calculations
5. If the fault is sliding, we compute the slip rate iteratively from eqs. (9) and (10) and evaluate an imposed state for tractions and velocities which satisfy these conditions.
6. With the final tractions and velocities calculated, we numerically calculate a correction flux to replace the respective neighbor fluxes in eqs. (7) and (8).

For full details on implementation of dynamic rupture with ADER-DG in SeisSol, we refer to Pelties et al. (2012); Wolf (2024).

Note that while some of the steps (3D-2D projection, Riemann solution) of the dynamic rupture implementation could again be implemented via small matrix or

tensor operations, the nonlinear equations eqs. (9) and (10) cannot be computed like this. We have to treat them separately when fusing the simulations.

4.3 Off-fault plasticity

In SeisSol, off-fault plastic deformation is implemented using a visco-elasto-plastic Drucker-Prager rheology (Andrews 2005; Dunham et al. 2011; Erickson et al. 2017; Templeton and Rice 2008; Xu et al. 2012). At each time step, we first compute the quantities Q_{ij}^{n+1} via eq. (8), i.e., assuming purely elastic behavior. To check whether the respective trial stresses σ^{trial} exceed elasticity limits and cause plastic deformation, we first need to transform the "elastic" solution Q_{ij}^{n+1}, which is represented via modal basis functions (orthogonal polynomials), into the corresponding nodal representation (based on Lagrange polynomials). We then evaluate at each nodal point a yield function f_{yield} that determines whether plastic deformation occurs. If $f_{\text{yield}}\left(\sigma^{\text{trial}}\right) < 0$, the material remains elastic at this point; otherwise, we trigger a modification of the stresses considering non-linear plastic deformation.

The yield function, which decides between elastic response or plastic deformation, is parameterized by two material-dependent properties: internal friction coefficient and cohesion – see Wollherr et al. (2018) for details on how these parameters influence the yield function. In the case of plastic deformation, we calculate an updated stress considering plastic yielding as

$$\sigma_{ij}^{n+1} = \kappa s_{ij}^{\text{trial}} + \sigma_m^{\text{trial}} \delta_{ij} \tag{11}$$

where κ is the yield factor, σ_m^{trial} is the mean normal stress in the trial state and s_{ij}^{trial} are the deviatoric stresses in the trial state. Again, we refer to Wollherr et al. (2018) for a detailed discussion. Finally, we need to convert the updated stresses σ_{ij}^{n+1} back, from the nodal to the modal basis.

In summary, the plasticity kernels are executed after the predictor and corrector steps, and calculate the yield functions and plasticity deformation – both non-linear functions – in every element. We require kernels for transferring the degrees of freedom between nodal and modal bases – which could be expressed as tensor instructions – and a non-linear kernel that modifies the stresses based on the (also nonlinear) yield function. For the latter kernels, we cannot use direct tensor contractions, but have to treat them separately when fusing the simulations (cf. section 5.2).

4.4 Optimized small matrix multiplication in SeisSol

The ADER-DG scheme, as summarized in section 4.1, can be implemented entirely via many small matrix multiplications (or tensor contractions). The matrix sizes primarily depend on the number of quantities in the vector $\mathbf{q}$ and on the number of polynomial basis functions in each DG element (e.g., 56 for convergence order 6). Table 1 gives an overview of selected matrices for convergence orders 3 and 6, respectively. The matrix or tensor operations are expressed as tensor contractions in Einstein summation notation, such as in eqs. (5), (7) or (8). The YATeTo package (Uphoff and Bader 2020) analyzes these tensor expressions, and generates optimized sequences of binary

Matrix	order	logical		non-fused		fused	
		size	storage	size	storage	size	storage
$\mathbf{Q}, \mathbf{I}$	3	10×9	dense	16×9	dense (144)	10×9	dense (90)
$(\hat{\mathbf{K}}^{\xi})^T$	3	10×10	sparse (7)	16×4	block (64)	16×4	sparse (64)
$\mathbf{A}^*$	3	9×9	sparse (24)	9×9	sparse (24)	9×9	sparse (24)
$\mathbf{Q}, \mathbf{I}$	6	56×9	dense	56×9	dense (504)	56×9	dense (504)
$(\hat{\mathbf{K}}^{\xi})^T$	6	56×56	sparse (294)	56×35	block (1960)	56×35	sparse (1960)
$\mathbf{A}^*$	6	9×9	sparse (24)	9×9	sparse (24)	9×9	sparse (24)

Table 1: Matrix storage size of commonly used matrices in the predictor kernel, for convergence orders 3 and 6. In column *logical*, *size* denotes the true matrix dimensions; *storage* indicates whether the matrix is dense or sparse (with number of non-zeros). The columns *non-fused* and *fused* show the storage layout used in the respective implementation. Matrix sizes can increase due to alignment ("padding") to vector lengths of SIMD registers (here for vector length 8); *block* indicates that a sparse matrix is stored as a dense block that contains all non-zeros.

general matrix-matrix multiplications (GEMM), elementwise products and reductions out of them. The small-GEMM kernels are delegated to hardware-optimized libraries: LIBXSMM (Heinecke et al. 2016) and PSpaMM. The latter is developed as part of the SeisSol package [3]. Both libraries optimize performance by generating either just-in-time assembly (LIBXSMM) or inline assembly kernels (LIBXSMM and PSpaMM) that are better suited for small matrices than standard BLAS libraries. The latter are typically designed for larger matrices, and therefore underperform for the matrix sizes in SeisSol, due to overheads in memory accesses, loop management, and unoptimized handling of matrices of particular (fixed) size (Heinecke et al. 2016; Dongarra et al. 2017; Masliah et al. 2016; Yang et al. 2021; Yao et al. 2021; Abdelfattah et al. 2016).

To showcase the workflow of YATeTo, LIBXSMM and PSpaMM, we consider an operation of the form $R_{ij} = K_{ik} Q_{kl} A_{lj}$ (using Einstein summation notation), which reflects a core pattern in the calculations in (5), (7), and (8). $\mathbf{K}$ denotes a comparably large and sparse matrix (cmp. table 1), which is constant over all elements; $\mathbf{A}$ denotes a sparse but small matrix, and $\mathbf{Q}$ and the result $\mathbf{R}$ are dense.

To start, YATeTo decomposes the given chain matrix product into a sequence of binary matrix products: a sparse-dense matrix multiplication $X_{il} = K_{ik} Q_{kl}$ followed by a dense-sparse matrix multiplication $R_{ij} = X_{il} A_{lj}$. Also, $\mathbf{X}$ and $\mathbf{R}$ are marked as dense, since $\mathbf{Q}$ is dense. Furthermore, YATeTo pads $\mathbf{Q}$, $\mathbf{X}$ and $\mathbf{R}$ to SIMD length (see table 1), to allow for more efficient code generation by LIBXSMM and PSpaMM.

In earlier work (Heinecke et al. 2014; Uphoff et al. 2017, e.g.), sparse-dense/dense-sparse multiplications were executed as dense-dense operations, if the sparsity (i.e., relative number of non-zero elements) of the involved matrices exceeded a certain threshold. We stress that for this work, we execute all sparse-dense/dense-sparse multiplications using sparse matrices and PSpaMM as code generator. Dense-dense operations are still executed via LIBXSMM.

[3] PSpaMM is available at https://github.com/SeisSol/PSpaMM.

Both LIBXSMM and PSpaMM generate hardware-optimized code (i.e., assembly instructions) that exploits SIMD instructions available on specific CPU architectures. The general implementation pattern for a dense-dense product (both matrices stored in row-major format) is to load consecutive elements of the left matrix into a SIMD register. The right matrix, in contrast, is loaded per-element, each element being broadcasted to all elements of a SIMD register to allow a subsequent SIMD operation ("fused multiply-add"; results accumulated in SIMD registers for the result matrix).

The difference in SIMD register handling for the two matrices requires different approaches to handling matrix sparsity efficiently: While dense-sparse products can be implemented by just removing all instructions and broadcasts of zero elements, sparse-dense products are much harder to implement efficiently.

However, for instructions such as $X_{il} = K_{ik}Q_{kl}$, we know that the stiffness matrix K_{ik} is identical for all elements, is thus stored only once and is expected to remain in low-level caches during computation. PSpaMM therefore employs a block-sparse (or padded sparse) approach with unrolling of element operations: the matrix is decomposed over SIMD blocks, which means that starting from a row-major dense layout, we split into blocks of SIMD length, and only keep those blocks that have at least one non-zero element. The compute kernels now omit instructions (and storage) for all blocks that are entirely zero, and for the remaining blocks mask out all known zero operations.

To summarize, we stress that the sparsity patterns of all matrices are known in advance and can be exposed to the code generators YATeTo and PSpaMM. Efficient handling of sparse matrices adopts padding approaches and dense blocks of SIMD length, which lead to additional operations being executed, but which are vastly superior to conventional sparse-matrix implementations in terms of time-to-solution (Heinecke et al. 2014, e.g.). We can evaluate the respective padding overhead via measuring the gap between "hardware" (executed SIMD instructions) and "non-zero" (i.e., absolutely necessary) operations. Fused ensemble simulations will reduce this gap, particularly (see section 5.1).

5 Fused simulations

We now discuss the implementation of fused simulations to accelerate large-scale earthquake modeling. The key idea to accelerate the wave propagation kernels follows previous work by Uphoff (2020) and Uphoff and Bader (2020). The central idea is to represent the degrees of freedom Q_{ij} of multiple simulations $s = 1, \ldots, S$ as a single rank-3 tensor Q_{sij} stored in each element. To update Q_{sij}, the ADER-DG scheme proceeds as described in section 4.1, essentially adding an additional dimension s to all numerical steps. The adjusted Cauchy–Kovalevskaya procedure (5) then reads

$$\begin{aligned} D^{p+1}_{sij} :=& \hat{K}^{\xi}_{im} D^{p}_{sml} A^{*}_{lj} + \hat{K}^{\eta}_{im} D^{p}_{sml} B^{*}_{lj} + \hat{K}^{\zeta}_{im} D^{p}_{sml} C^{*}_{lj} \qquad D^{0}_{sij} := Q_{sij}, \\ \text{and } I_{sij} =& \sum_{p=0}^{P} \frac{\Delta t^{p+1}}{(\delta+1)!} D^{p}_{sij}. \end{aligned} \tag{12}$$

The local corrector part (7) and the final corrector step (8) become

$$Q^{n+1,*}_{sij} = Q^{n}_{sij} + \tilde{K}^{\xi}_{im} I_{sml} A^{*}_{lj} + \tilde{K}^{\eta}_{im} I_{sml} B^{*}_{lj} + \tilde{K}^{\zeta}_{im} I_{sml} C^{*}_{lj} - \frac{1}{|J|} M^{-1}_{il} \left(\sum_{f=1}^{4} |S_f| \, F^{-,f}_{lm} I_{smn} \hat{A}^{+}_{nj} \right) \tag{13}$$

$$\text{and} \quad Q^{n+1}_{sij} = Q^{n+1,*}_{sij} - \frac{1}{|J|} M^{-1}_{il} \left(\sum_{f=1}^{4} |S_f| \, F^{+,i}_{lm} I^{f,-}_{smn} \hat{A}^{-}_{nj} \right). \tag{14}$$

For implementation, the key question is now whether to store the tensor Q_{sij} with s as the "fastest-running" index, i.e. as $(Q_{1,0,0}, \ldots, Q_{S,0,0}, \ldots, Q_{1,N,M}, \ldots, Q_{S,N,M})$ in each cell, or whether we store the fused data sets sequentially – resulting in a cell-local data layout $(Q_{1,0,0}, \ldots, Q_{1,N,M}, \ldots, Q_{S,0,0}, \ldots, Q_{S,N,M})$. (Here, N denotes the number of basis functions and M the number of quantities.) As we describe in section 5.1, having s as the "fastest-running" index is favorable for the fused wave propagation kernels, moving from batched matrix to batched tensor kernels. In contrast, as outlined in section 5.2, the dynamic rupture and plasticity computations are non-linear, cannot be expressed via batched matrix or tensor kernels alone, and are thus implemented via the sequentialized data structure.

5.1 Fusing the wave propagation kernels

As in section 4.4, we consider an example expression from eq. (13), which we write as $R_{sij} = K_{ik} Q_{skl} A_{lj}$. Again, YATeTo will decompose this tensor operation into two steps, $X_{sil} = K_{ik} Q_{skl}$ and $R_{sij} = X_{sil} A_{lj}$. Here, Q_{skl} as well as R_{sij} and X_{sil} are dense tensors of size $S \times N \times M$, whereas K_{ik} and A_{lj} are sparse matrices.

Next, YATeTo transforms the first operation into a loop (l being the loop variable) over GEMM operations: fixing l and contracting over k, each $X_{sil} = Q_{skl} K^{T}_{ki}$ becomes a dense-sparse matrix multiplication, executed by PSpaMM. If S is selected as a multiple of the SIMD length, this does not cause any sparse padding penalty (s being the fastest-running index) – in contrast to the non-fused situation.

The second operation, $R_{sij} = X_{sil} A_{lj}$, can be computed by a single dense-sparse GEMM that contracts over l and collapses the s and i dimensions. PSpaMM executes this dense-sparse multiplication as described in section 4.4, which does not cause any padding due to sparsity of A_{lj}, and also avoids padding in X_{sil} and R_{sij}, as the collapsed row index si is a multiple of the SIMD length (S).

This "loop over GEMM"-implementation for $X_{sil} = Q_{skl} K^{T}_{ki}$ and collapsed dense-sparse-GEMM implementation for $R_{sij} = X_{sil} A_{lj}$ has the following impacts on kernel performance (always assuming that S is chosen a multiple of the SIMD length):

- We avoid "dense padding" to SIMD length for the tensor of unknowns Q_{skl}: for example, assuming a SIMD length of 8 and order 3, $\mathbf{Q}$ is padded for serial executions from size 10×9 to size 16×9 (compare table 1). Fused simulations use an $S \times 10 \times 9$ tensor without any padding. We thus reduce the costs for fetching the Q_{kl}/Q_{skl} from main memory – by a factor of $16/10 = 1.6$ in the given example.

- We avoid "sparse padding" for the operation $X_{sil} = Q_{skl} K^T_{ki}$: We thus increase the fraction of non-zero operations compared to hardware operations.
- Especially at low order, sparse and dense padding together increase the arithmetic intensity of the kernels.
- Collapsed loops in the operation $R_{sij} = X_{sil} A_{lj}$ and differences in register usage for unrolled matrix vs. tensor operations may lead to further performance improvements.
- On the other hand, fused simulations might compromise one of the central assumptions in our ADER-DG optimization, which is that element-local operations fit into low-level caches. Especially for high polynomial order, this assumption will be violated if S is chosen too large. For example, at order 6 and with $S = 16$, the tensor Q_{skl} is already of size $16 \times 56 \times 9$, equivalent to 8,064 elements and approximately 64 KB for double precision, which already exceeds the size of the level-1 caches of our architectures (compare section 7.1).

5.2 Fusing the non-linear kernels

As indicated in section 4.2 and section 4.3, the dynamic rupture and plasticity kernels are non-linear and we cannot directly utilise the tensor structure to "fuse" the simulations. We therefore change the data layout to compute the non-linear kernels from the format Q_{sij} to Q_{ijs} i.e., from $(Q_{1,0,0}, \ldots, Q_{S,0,0}, \ldots, Q_{1,N,M}, \ldots, Q_{S,N,M})$ to $(Q_{1,0,0}, \ldots, Q_{1,N,M}, \ldots, Q_{S,0,0}, \ldots, Q_{S,N,M})$. This enables a clear outer loop over the simulation index s (i.e., over the S problems), in which each Q_{ij} tensor is processed independently for dynamic rupture and plasticity. Once the non-linear kernels are finished, the data structures are packed back to their original shape, i.e., to Q_{sij} format. The algorithm is summarized as:

- initialize a temporary data structure to store the transposed data structure
- copy the data structure into the temporary data structure $\hat{\mathbf{Q}}$ with ordering $\hat{Q}_{ijs}$
- perform for each simulation index s in 1 to S
  - pick the data for simulation s, i.e., $(\hat{Q}_{s,0,0}, \ldots, \hat{Q}_{s,N,M})$
  - perform the non-linear kernel on this data
- copy the computed data from the temporary data structure into the original data structure with format Q_{sij}.

In the case of dynamic rupture, this algorithm is executed for all faces with a dynamic rupture interface. For plasticity, it is executed for all elements.

5.3 Validation of correctness

We validate the correctness of the fused-simulations implementation against the SCEC TPV13 community benchmark scenario, as introduced by Harris et al. (2018)[4]. TPV13 is a 3D earthquake dynamic rupture benchmark with off-fault plastic yielding. It

[4] See detailed description by the SCEC/USGS Spontaneous Rupture Code Verification Project (https://strike.scec.org/cvws/) on https://strike.scec.org/cvws/download/TPV12_13_Description_v6.pdf.

prescribes a 60-degree dipping planar, normal fault ($30 \times 15\,\text{km}$) embedded in a homogeneous halfspace. The fault reaches the Earth's surface and extends up to ≈13 km in depth. The off-fault rheology is non-associative Drucker-Prager visco-plastic (compare section 4.3). Depth-dependent initial stress conditions are prescribed on and off the fault. The linear slip-weakening friction law is used. Strongly super-shear rupture conditions are assumed. Rupture is initiated in a square nucleation area (side length 3 km), which is centered at $(0, -12)\,\text{km}$ (in fault coordinates) at a depth of ≈10.4 km.

We compute on a mesh with ≈1.4 million elements, with ≈25.5 thousand elements adjacent to the fault (i.e., computing dynamic rupture). For the fused simulations, we vary the coefficient μ_s for static friction inside the nucleation patch. For one simulation, we set it to obtain no rupture at all. To verify that fused simulations do not produce major differences in the results, we compared particle velocities at different receiver locations for fused and single simulations. Figure 1 plots the velocity v_1 at receiver $(0.0, 826.795, -300.0)\,\text{m}$, when computing in single precision. Figure 2 shows the corresponding absolute errors for simulation numbers 3 and 4 in single precision. In single precision, we observe an error in the order of 10^{-4} for this quantity. We repeat the validation using double precision, which brings the maximum absolute error down to the range of 10^{-14}. We plot the errors with double precision for simulations 3 and 4 in Figure 3.

We observe close agreement of the solutions, with negligible floating-point errors in case of double precision. The errors are higher (in the range of 10^{-4}) with single precision, which we attribute to a high sensitivity of dynamic rupture simulations w.r.t. the selected precision. Differences in rounding can be due to the different execution order of operations and the different intermediate rounding of floating-point values in sequences of SIMD operations. However, our results indicate that the fusion of simulations does not have a significant impact on accuracy.

6 Launching fused ensemble simulations in UQ workflows with UM-Bridge

As outlined in section 2, we rely on UM-Bridge (Seelinger et al. 2025) to implement complex UQ workflows with SeisSol. UM-Bridge is a software interface between UQ algorithms (a *client*) and numerical simulators (a *model*), and enables any client to request simulation runs for a UQ workflow. SeisSol provides an UM-Bridge wrapper, and UM-Bridge schedules simulation runs on a compute cluster in a way that is entirely transparent to the client side.

This section describes how we extend this setup to support fused simulations. The key challenge is that the UQ side remains entirely oblivious to fusing simulations, making fused simulations a drop-in replacement for simulations within *any* UM-Bridge enabled workflow that allows appropriate parallelism.

6.1 Batching UM-Bridge model queries

In order to transparently form batches of parameters for fused simulations, we introduce a microservice-style component in between UQ and simulator. To the UQ side,

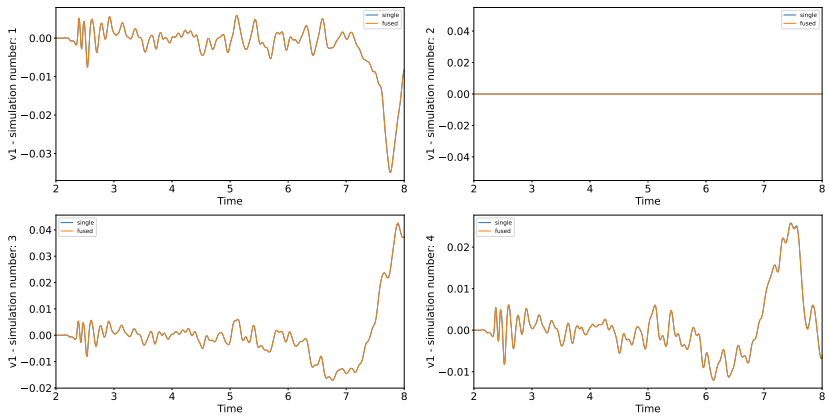


Figure 1: Validation of TPV13 results using single precision. Each plot compares the results of the fused simulations with their serial counterparts, revealing no visible discrepancy. The parameters of Simulation 2 (top right) were set such that there is no rupture, to validate this case, as well. The time axis is clipped before 2 s of simulation time, as no waves are observed there, yet.

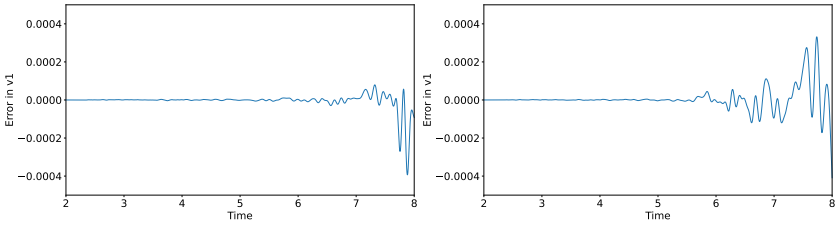


Figure 2: Absolute errors between the fused and serial simulations for simulations 3 and 4 of fig. 1, using single precision.

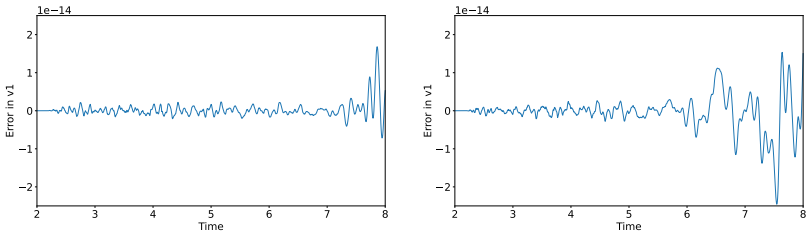


Figure 3: Absolute errors between the fused and serial simulations for simulations 3 and 4 of fig. 1, using double precision.

this batcher[5] appears as an UM-Bridge model taking a single parameter vector and returning a single model output, and is therefore entirely equivalent to non-fused simulation runs. Internally, it collects batches of parameters, passes entire batches to the UM-Bridge HPC load balancer (which spins up SLURM jobs with SeisSol as needed), unpacks batched model outputs, and returns them individually to the respective client request. The simulator is only concerned with processing one parameter batch at a time, and is therefore oblivious to how parameter batches are assembled or how the UQ side is parallelized.

Importantly, in case the number of incoming requests does not amount to the batch size expected by SeisSol within a sufficiently short time, we pad the existing request parameters to meet the full batch size. While this guarantees timely executions of simulations, padding of course implies wasting some of the fused simulations in a batch. Collection and execution of batches is fully thread-parallel, making sure the batcher is permanently responsive.

The resulting design enables dynamic and asynchronous grouping of parameter evaluations for fused ensembles with improved computational efficiency, while maintaining compatibility with the UM-Bridge API and minimal intrusion on the simulator code. Additionally, we include retry handling in the batcher to mitigate potential instabilities in large-scale HPC environments.

6.2 SeisSol UM-Bridge server

To enable seamless integration with UM-Bridge and support for fused simulations, we extended SeisSol's UM-Bridge server. When a client request (from MLDA in our case) arrives, the wrapper generates input files for SeisSol through the Inja templating engine (https://pantor.github.io/inja/), executes the simulation, and returns misfit values computed by a Python postprocessing script.

Depending on the simulation configuration (e.g., order 3 or 4), the wrapper dynamically selects the appropriate binary for either fused or serial simulations. Environment variables and system-level settings are adjusted depending on the computing platform, for example with specific configurations tailored for the CPU architecture. The generated input files (incl. parameters, fault details, etc.) are placed into a scratch directory, identified by the SLURM job ID to separate files of concurrent runs. The misfit computation to calculate the loglikelihood for the MLDA algorithm is performed externally via a Python script, and results are captured and parsed from standard output.

This setup enables efficient and scalable black-box evaluations of SeisSol via UM-Bridge clients and lets us separate fused simulations with unfused simulations for different configurations. The resulting workflow proceeds as follows:

1. The client compatible with UM-Bridge (UQ) sends simulation parameters to the batcher.
2. The batcher collects the simulation parameters, batches them as per the configuration and sends the vector to the UM-Brige server.
3. The UM-Bridge HPC loadbalancer receives the request and spans SLURM jobs.
4. The UM-Bridge wrapper for SeisSol inside the SLURM jobs processes the request:

[5]The implementation is available at https://github.com/SeisSolUQ/Batcher/blob/main/batcher.py.

- Produce input files (via template generation) based on the received parameters.
- Execute the appropriate SeisSol binary on the generated files.
- Evaluate the misfit (via a Python script) between the written simulation output and the observed data.

5. The resulting misfit values are sent back to the batcher.
6. The batcher disaggregates the results and returns them to the client.

The code for the UM-Bridge server with SeisSol can be found at https://github.com/SeisSolUQ/Seis-Bridge/blob/main/ridgecrest/ridgecrest-server.cpp

7 Performance of fused earthquake simulations

7.1 Architectures

For our experiments and performance studies, we used three different CPU-based supercomputers with different architectures, with the following specifications:

Vista[6] (Texas Advanced Computing Center, TACC), in its Grace-Grace subsystem, offers 256 nodes equipped with an NVIDIA Grace-Grace CPU Superchip that combines two Arm CPUs with 72 cores, clocked at 3.4 GHz and including 64 KB of L1 cache. Each node has 237 GB of LPDDR DRAM memory with a main memory bandwidth of 850 GiB/s and a peak performance of 7.1 TFlop/s (using double precision).
Frontera[7] (TACC) combines 8,368 compute nodes, each equipped with two Intel Xeon Platinum 8280 (Cascade Lake) CPUs, each with 28 cores, clocked at 2.7 GHz and including 32 KB of L1 cache. Each node has 192 GB of DDR-4 memory with a main memory bandwidth of 282 GB/s and a peak performance of 4.8 TFlop/s (double precision).
SuperMUC-NG[8] (Leibniz Supercomputing Centre, LRZ), in its Phase 1 ("thin" nodes), offers 6,336 compute nodes equipped with two Intel Xeon Platinum 8147 (Skylake) CPUs, each with 24 cores, clocked at 2.7 GHz and including 32 KB of L1 cache. Each node has 96 GB of memory with a main memory bandwidth of 102.4 GB/s and a peak performance of 4.15 TFlop/s (double precision).

While the SIMD width of Vista's Arm CPUs is 16 bytes (2 double-precision values per SIMD register), the Intel CPUs on Frontera and SuperMUC-NG use a SIMD width of 64 bytes, equivalent to 8 double-precision values. We can efficiently fuse 2 double-precision simulations, or any multiples of 2, on Vista. In contrast, on Frontera and SuperMUC-NG, we are restricted to multiples of 8, i.e., we can fuse 8, 16, ... double-precision simulations without wasting performance.

The results on the Frontera supercomputer were partly obtained during a *Texascale Day* that allowed for full-machine runs. During most of the runs, we noticed significant performance fluctuations between compute nodes on Frontera, which affected the performance measurements. We comment on the details at the respective experiments.

[6] https://docs.tacc.utexas.edu/hpc/vista/
[7] https://tacc.utexas.edu/systems/frontera/
[8] https://doku.lrz.de/supermuc-ng-10745965.html

7.2 Performance evaluation of fused wave-propagation kernels

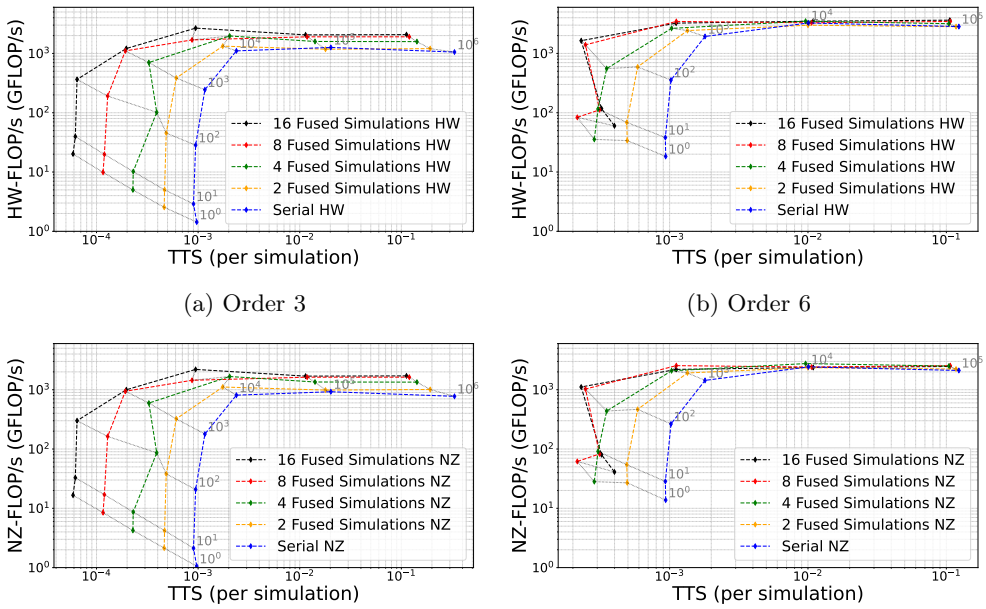
As a first step, we examine the influence of fused simulations on the performance-dominating kernels for seismic wave propagation. SeisSol-Proxy is a mini app that executes the key SeisSol compute kernels on arbitrary data to allow detailed performance studies. In fig. 4, we compare the floating-point operations achieved per second (FLOP/s) achieved for single vs. fused simulations on a single Grace-Grace node of Vista. While we consider all FLOP/s issued in the hardware units ("Hardware FLOP/s": HW-FLOP/s) in subfigures (a) and (b), we display the actual FLOP/s contributing towards the solution ("Non-Zero FLOP/s": NZ-FLOP/s) in subfigures (c) and (d). The NZ-FLOP/s exclude any operations from the HW-FLOP/s done due to SIMD register padding or handling a sparse as dense matrix. Hence, HW-FLOP/s consider the actual number of computations performed by the computer, while NZ-FLOP/s are the number of useful computations done.

We executed only the wave propagation kernels (local element updates and neighbor kernels) using 1 MPI rank per node and 143 OpenMP threads per rank, which leaves one OpenMP thread per rank to be used as a communication thread. We pinned the OpenMP threads to cores in a 1:1 fashion, using the `OMP_PLACES=cores(143)` option with a `OMP_PROC_BIND=close` binding. We plot the achieved HW-FLOP/s and NZ-FLOP/s vs. the time to solution required to update the elements, where we increase the number of elements by a factor of 10 from each test run to the next.

We observe for all simulations (fused and non-fused) that the time-to-solution first stagnates, as the launch of kernels dominates the computing time. Note also that OpenMP parallelism (here with 143 threads) can only be exploited for more than 100 elements. All simulations then plateau, eventually reaching the full kernel performance for a large ($> 10^4$) number of elements. The fused-simulation kernels plateau earlier, though, and at a higher performance than serial simulation. For discretization order 3, we observe an increase of HW-FLOP/s by a factor of up to 1.97 and NZ-FLOP/s by a factor of up to 2.21 for 16 fused simulations. The improvement is much smaller for order 6, with an increase of HW-FLOP/s of 1.28 and NZ-FLOP/s of 1.17.

These results indicate that in addition to the five aspects already mentioned in section 5.1, there are two further issues that impact performance: Due to the short SIMD length of the Grace-Grace superchip, dense padding and sparse padding should lead to a much smaller effect than the observed $\approx 2\times$ speedup for the memory-bound situation of order 3. We attribute the larger speedup to memory latency effects, i.e., that larger blocks of memory (consisting of several fused element-local data structures) are transferred more efficiently on Grace-Grace, leading to an improved memory access pattern. Second, we observe from the gray lines in fig. 4 that for a fixed cluster size, the improvement is much higher for small clusters: for order 3, the NZ-FLOP/s increased by a factor of 14.43 for 10^2 and 5.57 for 10^3 elements (comparing serial execution with 16 fused simulations). Here, the launch overhead for OpenMP kernels comes into play, and we have to consider that with 143 threads, each thread will only have a few elements assigned. Launch times are thus large compared to processing times, and launch overheads are amortized when using many fused simulations.

The performance benefit for small clusters is particularly relevant when running SeisSol with local time stepping (LTS). On realistic meshes with aggressive mesh



(a) Order 3

(b) Order 6

(c) Order 3

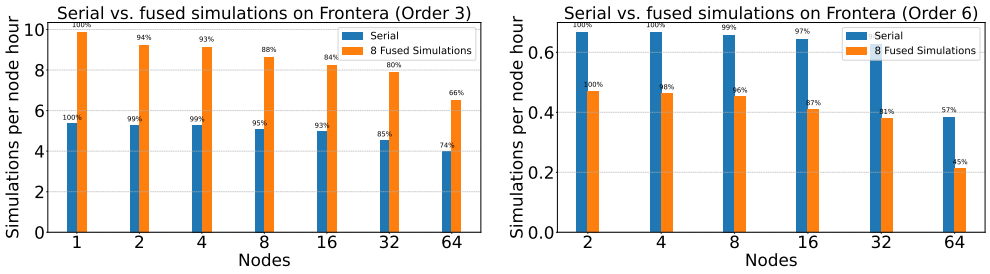
(d) Order 6

Figure 4: HW-FLOP/s (plots a and b) and NZ-FLOP/s (c and d) plotted against the time-to-solution (TTS) per-simulation, when running the SeisSol proxy on Vista (Grace-Grace) with and without fused simulations. We increase the problem size from 1 to 10^6 elements (each step increasing cluster size by a factor of 10; grey lines connect identical problem sizes). For small element counts, the kernel launch time dominates (plots growing vertically), whereas for larger element counts, the throughput dominates (plots plateauing).

refinement and complicated geometries, we often encounter grid cells with bad element shapes that enforce a small local time step. SeisSol's LTS algorithm combines elements with similar time step limits into LTS clusters, which are updated with a uniform time step (Breuer et al. 2016; Uphoff et al. 2017). Figure 4 illustrates that small LTS clusters suffer from a substantial performance reduction.

7.3 Scalability for medium-size problems – fused TPV-13 simulations

Next, we test the performance and scalability of fused simulations on Frontera and Vista. We first focus on the TPV13 benchmark (cmp. section 5.3) and thus a comparably small problem size, to reflect the behavior of model evaluations that form the bulk of executions in an MLDA workflow. We run eight different simulations with varying coefficient μ_s of static friction, first as serial and then as fused simulations, and compare the execution times and speed-up for an increasing number of nodes. We use two



(a) Order 3

(b) Order 6

Figure 5: Simulations per node hour achieved for the TPV-13 benchmark on Frontera, comparing single (blue) with eight fused (orange) simulations at discretization orders 3 and 6. The annotated value above each bar indicates the parallel efficiency (relative to 1 node for O3, and 2 nodes for O6).

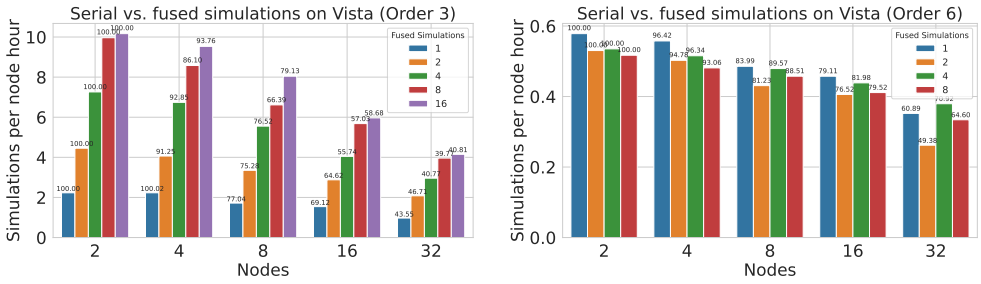
l

MPI ranks per node (i.e., one rank per socket) with 27 OpenMP threads per rank for Frontera and 71 threads per rank for Vista, hence, again reserving one thread per rank for the communication. For pinning threads to cores, we use the approach described in section 7.2. As a performance metric, we use "simulations per node hour", which is the metric to optimize for large workflows: it specifies how many simulation runs we can execute for a given amount of compute resources. In our performance and scaling tests in section 7.3 and 7.4, we ignore the mesh reading, model initialization and I/O, and compute the "simulations per node hour" as the wall-clock time required for the simulation itself divided by node count and time to solution.

In fig. 5, we plot the simulations per node hour achieved on Frontera for discretization orders 3 and 6. For order 3, we observe a speedup of 45% on a single node, which slightly reduces for increasing node count, reaching a speedup of 38% on 64 nodes. For this memory-bound situation, the speedup is in good agreement with what we could expect from reduced memory transfers, due to avoiding dense padding. For order 6, we observe a drop in performance for fused simulations, which we attribute to element-local data structures no longer fitting into the L1 cache. For both orders, we see a slightly worse parallel efficiency for the fused simulations, which we attribute to the larger communication volume for fused simulations. In fig. 6, we plot the simulations per node hour achieved on Vista, again for orders 3 and 6. For order 3, the performance improvement for fused simulations is much bigger than on Frontera. For 16 fused simulations, we get a 4.6× speedup on 2 nodes, and still a 4.3× speedup on 32 nodes (despite the suboptimal scalability on Vista). Fused simulations are, however, again slower with order 6, though by a smaller margin – profiting from smaller padding overhead (shorter SIMD length) and larger L1 cache.

7.4 Scalability for large problems – Kahramanmaraş scenario

We now test the performance and scaling of fused simulations for a large-scale production run. As a scenario, we simulate the 2023 Kahramanmaraş, Turkey, M_w 7.8–7.7



(a) Order 3

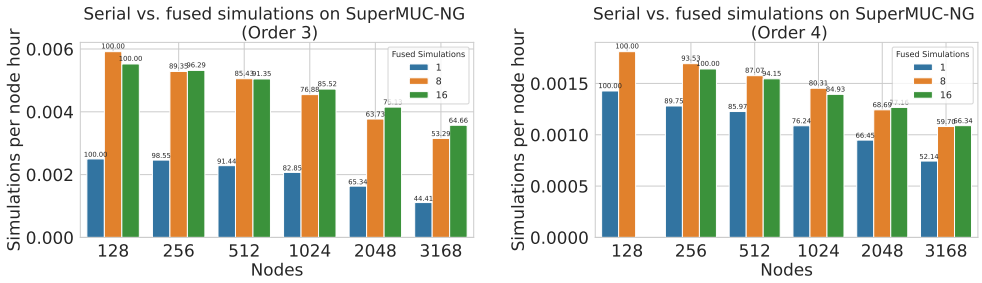
(b) Order 6

Figure 6: Extrapolated simulations per node hour achieved for the TPV-13 benchmark on Vista (Grace-Grace), comparing single execution (blue bars) with 2, 4, 8 or 16 fused simulations (all for discretization orders 3 and 6). The scaling tests have been performed for 0.1 s of simulated time. The number of simulations per node hour is extrapolated to consider a simulation time of 8.0 s. The bars are annotated with the parallel efficiencies for every fused simulation configuration.

earthquake doublet, using models by Gabriel et al. (2023) and Jia et al. (2023), which involved complex rupture dynamics across numerous fault segments. This scenario models elastic wave propagation, with Drucker-Prager plastic deformation and a linear slip weakening friction law (cmp. sections 4.2 and 4.3).

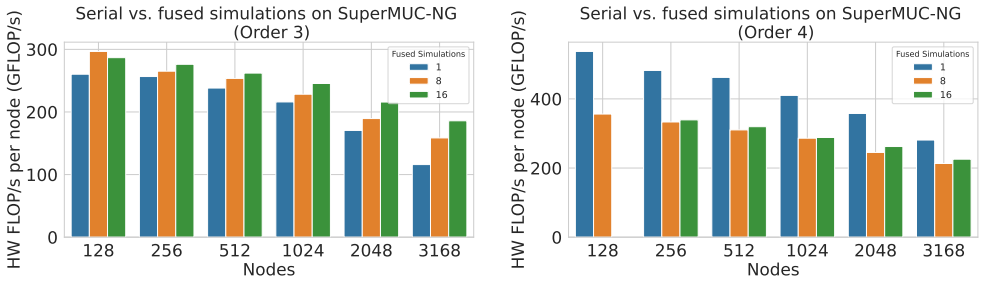
We use a mesh with approximately 175 million elements, which leads to 15.75 billion degrees of freedom at discretization order 3, and 31.5 billion degrees of freedom at order 4. For the performance study, we truncated the simulation to the first 1.0 s of the event and extrapolated the number of simulations per node hour accordingly. We tested it on the SuperMUC-NG with one MPI rank per node with 94 OpenMP threads per rank (2×47 due to hyperthreading on SuperMUC-NG), again reserving one core per rank for the communication and pinning it to the last core as described in section 7.2. We plot the extrapolated simulations per node hour for orders 3 and 4 in fig. 7. Since the configuration with 128 nodes with order 4 and 16 fused simulations did not fit in memory, this data is missing in fig. 7b. On SuperMUC-NG, fused simulations with both 8 and 16 fused-simulations configurations perform better than serial simulations for both orders 3 and 4.

Figures 8 and 9 show the HW-FLOP/s measurements and the equivalent NZ-FLOP/s measurements for the same configurations. For order 3, we observe that the HW-FLOP/s are higher for fused simulations by a factor of around 1.1. For order 4, however, serial simulations have better HW-FLOP/s performance, up to a factor of 1.5 for lower-node configurations. Nevertheless, the NZ-FLOP/s are higher for fused simulations for both orders. On 128 nodes, we observe a 2.3× improvement in NZ-FLOP/s for order 3, and 1.2× for order 4. This is due to the lack of padding and better sparsity utilization for lower orders of the simulation, in the case of fused simulations. For 3168 nodes (half of the SuperMUC-NG machine), 16 fused simulations reached a peak performance of 714.5 TFLOP/s with a non-zero performance of 549.3 TFLOP/s.



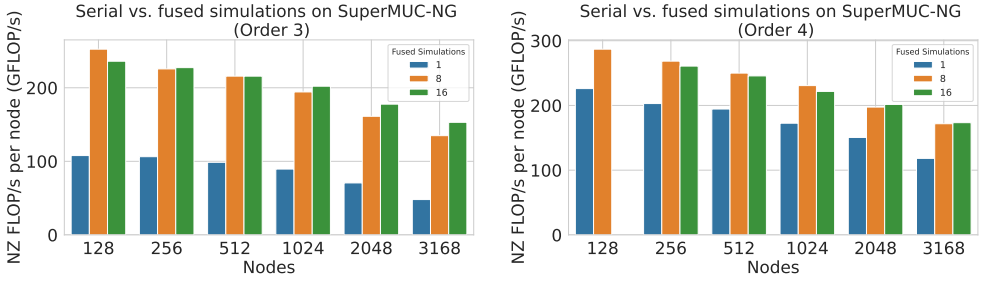
(a) Order 3 (b) Order 4

Figure 7: Extrapolated simulations per node hour achieved for the Turkey scenario on SuperMUC-NG, comparing single execution (blue bars) with fused simulations with 8 or 16 fused simulations (all for discretization orders 3 and 4). The bar plots are annotated with the parallel efficiencies. The scaling test was performed for a simulation time of 1 s. Simulations per node hour are extrapolated for a simulation time of 150 s.



(a) Order 3 (b) Order 4

Figure 8: HW-FLOP/s per node measured on SuperMUC-NG for the Turkey Kahramanmaraş scenario with 175 million elements



(a) Order 3 (b) Order 4

Figure 9: NZ-FLOP/s per node measured on SuperMUC-NG for the Turkey Kahramanmaraş scenario with 175 million elements

Nodes	Fused simulations	Simulations per 10^3 node hours	HW FLOP/s [PFLOP/s]	NZ FLOP/s [PFLOP/s]	Relative Performance
3584	1	3.00	1.22	0.467	1
	8	5.66	1.02	0.870	1.88
	16	4.92	0.918	0.756	1.64
3072	1	3.34	1.17	0.448	1
	8	5.73	0.886	0.745	1.72
	16	5.77	0.924	0.760	1.73
2560	1	3.48	1.01	0.386	1
	8	5.10	0.657	0.559	1.47
	16	6.13	0.817	0.673	1.76

Table 2: Performance comparison of fused and serial simulations for the Turkey scenario on Frontera for order 3. "Relative performance" states the ratio of simulations per node hour between fused and serial execution.

Nodes	Fused simulations	Simulations per 10^3 node hours	HW FLOP/s [PFLOP/s]	NZ FLOP/s [PFLOP/s]	Relative Performance
3584	1	1.84	2.49	1.05	1
	8	1.58	1.12	0.904	0.86
	16	1.53	1.14	0.874	0.83
2560	1	1.86	1.80	0.757	1
	8	1.69	0.856	0.690	0.91
	16	1.60	0.850	0.653	0.86

Table 3: Performance comparison of fused and serial simulations for the Turkey scenario on Frontera for order 4. "Relative performance" states the ratio of simulations per node hour between fused and serial execution.

We performed the same scaling test on the Frontera supercomputer. Due to performance fluctuations (cmp. section 7.1), we only obtained reliable results for selected setups, which we therefore present in table form – see table 2 and table 3. For order 3, we observed a peak non-zero performance of 870 TFLOP/s (on 3584 nodes) with 8 fused simulations and a maximum performance gain of 1.88× relative to serial execution. For order 4, we observed the peak non-zero performance with serial simulations, at 1.05 PFLOP/s on 3584 nodes. Fused simulations only reached a relative performance between 0.83 and 0.91.

Overall, the performance results for the Kahramanmaraş scenario agree well with that for the smaller TPV13 benchmark. The gains due to fused simulation are slightly higher, especially at order 3, which we attribute to having a more complex LTS-cluster situation, with more time-step-size levels and smaller LTS clusters occurring. For the intended UQ workflows, presented in section 8, we note that on the Frontera supercomputer, it is beneficial to run fused simulations at order 3, but already from order 4, the serial simulations perform better.

8 Fused simulations in UQ workflows

In this section, we demonstrate that fused ensemble simulations can yield performance gains in actual, complex UQ workflows. Specifically, we perform Bayesian parameter inference for a complex earthquake model using the Multilevel Delayed Acceptance (MLDA) method (Lykkegaard et al. 2023; Kruse et al. 2024) outlined in section 2.

8.1 Bayesian inversion through parallel MLDA

As a Bayesian approach, MLDA considers inverse problems where the available data are noisy measurements of the output of a PDE model run with some unknown parameter θ. It combines prior knowledge about the parameter with a likelihood derived from the assumed observation noise to obtain a posterior distribution π. This posterior distribution summarizes which parameter values are more likely with the observed data and prior information. In general, Markov Chain Monte Carlo (MCMC) methods generate a sequence of correlated parameter samples by proposing candidates and accepting or rejecting them based on how well they fit the data. Because each acceptance decision requires evaluating the model-based likelihood (i.e., a PDE solve) and many samples are usually needed to obtain reliable and effectively independent draws, standard MCMC methods often become very expensive. Multilevel MCMC algorithms, such as the MLDA algorithm, can significantly decrease the high costs of such evaluations, by employing a hierarchy of models with different accuracy-cost tradeoffs (Dodwell et al. 2019).

In this work, we restrict ourselves to an MLDA hierarchy of two levels with corresponding posterior densities π_1 and π_2. The coarse level (using a lower polynomial order in SeisSol) is cheaper to compute, but only yields coarse approximations of the posterior. The fine level (higher order in SeisSol) is more expensive, but more accurate – and the fine level also determines the final computed posterior: $\pi = \pi_2$. MLDA generates high-quality proposals on the fine level by spawning an MCMC chain on the coarse level at the current state, and using the final coarse-level sample as a proposal for the fine level. MLDA is thus inherently sequential. A method of parallelization, though, is to generate multiple independent MLDA chains and combine their results. This is valid since all chains sample the same posterior; however, since each chain requires a burn-in period and mixing, parallel chains have diminishing returns. In Kruse et al. (2024), we introduced within-chain parallelization through prefetching to mitigate that limitation. For this particular inversion study, we restrict ourselves to parallel chains only, i.e., without any prefetching, as SeisSol scales well enough for us not to reach the regime where prefetching pays off. Following the performance results from section 7.4, we batch coarse-level model evaluations from parallel MLDA chains into fused ensemble, whereas we use serial simulations for fine-scale runs.

8.2 Bayesian parameter inversion – Searles Valley earthquake

We consider dynamic rupture simulations of the 2019 M_w 6.4 Searles Valley earthquake (fig. 10a), which involves rupturing a complex fault system comprising conjugate non-planar segments. The simulations integrate nonlinear frictional interactions on the fault (Dunham et al. 2011) and the propagation of waves generated from the breaking

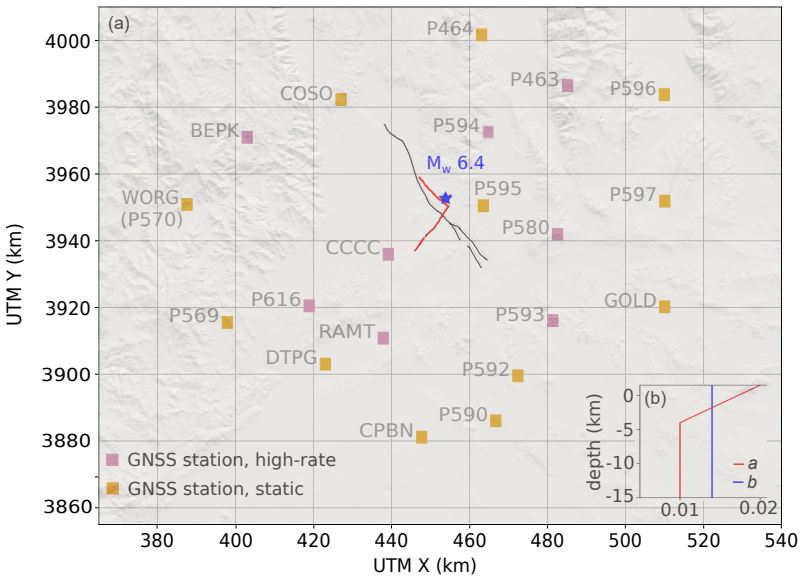
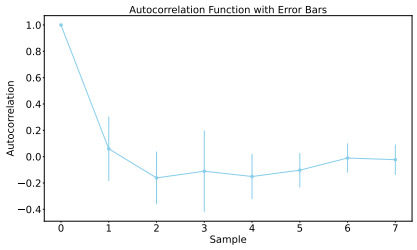


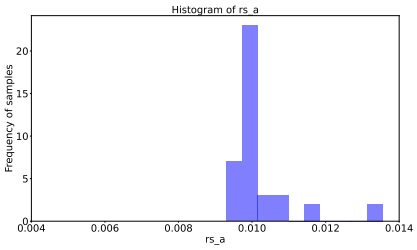
Figure 10: Data and model setup for the inversion with fused simulations. (a) Map view of the GNSS stations that constrain the inversion. The stations are located within a distance of 100 km to the epicenter (the blue star) of the M_w 6.4 Searles Valley earthquake. We mark the fault trace ruptured by the earthquake with a solid red curve. The pink rectangles denote GNSS stations where processed displacement time histories are available from Melgar et al. (2020); the orange rectangles are stations where we only use their co-seismic static displacement from Floyd et al. (2020). (b) Variations of the friction parameters a and b with depth.

fault. The computational domain was discretized with a mesh size of 4.02 million elements, ensuring adequate resolution of both the fault zone processes and seismic wave propagation (Kruse et al. 2024). Among the friction parameters, we select one physically important parameter $a - b$ that controls the spontaneity of the rupture dynamic (Dunham et al. 2011). In rate-and-state friction laws (Dieterich 1979; Ruina 1983), a and b jointly determine how the friction strength depends on the slip rate between two sides of the fault (direct effect vs. evolution effect). Earthquakes nucleate on faults with $a - b < 0$ and stop on the part of faults with $a - b > 0$ due to unconsolidated fault gouge (Kaneko and Fialko 2011). In our setup, we keep $b = 0.014$ fixed and vary the depth-dependent a in the inversion to perturb the value of $a - b$. Similar to Taufiqurrahman et al. (2023), we change a at the depths below -4 km between $a = 0.004$ and $a = 0.014$ and linearly interpolate a between -4 km and $a = 0.02$ at depth 0 km (fig. 10b). We use the near-fault surface displacement data at the global navigation satellite system (GNSS) stations (Melgar et al. 2020; Floyd et al. 2020) to constrain a in the inversion.

We use the MLDA algorithm with 8 parallel chains to invert for the parameter a, with convergence orders 3 and 4 as the two levels. We executed the workflow on the Frontera supercomputer, where fused simulations perform better with order 3



(a) Evolution of autocorrelation of the log-likelihood across different chain samples. The dots show the mean value across samples, and the dashes span the variance observed across different parallel chains.



(b) Histogram of fine-level samples drawn across all samples of all parallel chains. For each bin we give the number of samples drawn within the corresponding parameter range.

Figure 11: Behavior of the MLDA algorithm for the Ridecrest inversion study.

Metric	**Fused Simulations**	**Serial Simulations**
Simulation jobs for order 3	48	342
Actual parameter runs for order 3	384 (48×8)	342
Simulation jobs for order 4	72	72
Node hours for order 3	645	863
Node hours for order 4	415	437
Node hours in total	**1060**	**1300**

Table 4: Comparison of resource usage between serial and fused simulations for the MLDA workflow.

(cmp. section 7), but serial simulations perform better with order 4. Hence, we fused 8 simulations for all coarse-level runs, but used serial runs for the finer level. We used a sub-sampling rate of 8, which means that for every sample of a chain, we run approximately 8 coarse-level runs, such that the number of lower-order runs is significantly higher than the higher-order runs. The resulting autocorrelation oscillates around zero after around 8 samples. We plot the variation in autocorrelation across the parallel chains in fig. 11a and the histogram of the sample distribution in the parameter space $[0.004, 0.014]$ in fig. 11b. As the initial 3 samples of each chain had high autocorrelation, we treat them as burn-in and discard them. As we are inverting for a single parameter only in this example workflow, we only compute relatively few samples. Nevertheless, the results are in good agreement with the parallel work by Niu (2025).

In table 4, we report the node hours required on Frontera[9]. We used 32 nodes and executed SeisSol with 2 MPI ranks per node (one per CPU), such that MLDA chains were only parallelized via fused simulations (for coarse-level runs). The workflow with fused simulations reduces the node hours by around 18% compared to using serial

[9] As we observed performance fluctuations across different runs on Frontera, we ran the workflow thrice, and used the final runs, which were less varying.

simulations for all model evaluations. This improvement is lower than anticipated in comparison to results observed in section 7. This is due to the interaction of the MLDA algorithm, UM-Bridge, and SeisSol, which required multiple batches of fused simulations to be padded with nominal values. These are not used in the MLDA workflow, but are padded in the input vector of the fused simulations just to fulfill the constraint for fused simulations.

In an ideal scenario, all parameters evaluated with fused simulations would be used for inversion, resulting in 384 lower-order simulations. If we extrapolate the current serial simulations' node hours to compute the node hours required for 384 simulations, serial simulations would need 1406 node hours. With this, we can compute an ideal improvement in node hours consumed to be 24.5%. This ideal improvement includes the finer level runs with order 4, where we do not have a performance improvement across different workflows. For comparison, in section 7.3, we obtained an improvement of 42% in simulations per node hour for the TPV13 benchmark on Frontera for order 3.

9 Conclusions and outlook

We have implemented the concept of fused ensemble simulations for complex dynamic rupture earthquake simulations with off-fault plasticity in SeisSol. In the performance-dominant wave-propagation kernel, switching from matrix to tensor data structures leads to more efficient use of SIMD instructions for the underlying element-local dense-sparse and sparse-dense matrix multiplications. We also avoid padding in the dense quantities tensor, thus reducing required transfers from and to main memory. On the other hand, we need to re-arrange data structures on-the-fly for the dynamic rupture and plasticity kernels, and for large polynomial orders, element-local operations may fall out of cache. With additional effects from clustered local time stepping and granularity of memory transfers, we therefore observe widely varying performance benefit, or also loss of performance, depending on polynomial order, number of fused simulations, simulation scenario (mesh properties and size) and especially CPU architecture.

For smaller benchmark scenarios, we achieved improvements in simulations per node hour of up to 4.56× on the Vista supercomputer (using the NVIDIA Grace-Grace Superchip) vs. 1.83× on the Frontera supercomputer (using Intel Skylake CPUs), both for discretization order 3. For discretization order 6, we see a loss of performance (10% and 25% less) on both platforms. For a large production scenario, the 2023 Kahramanmaraş earthquake, we observed a 2.37× improvement in simulations per node hours for order 3, but diminishing improvement (1.32×) already for order 4, executed on the SuperMUC-NG supercomputer (Intel Skylake).

We have extended the UM-Bridge UQ and modeling interface by a batcher component to combine multiple subsequent simulation runs into fused ensembles, in a way that is transparent to the UQ algorithms. Based on UM-Bridge and a respective SeisSol wrapper, we demonstrated the benefit of fused-ensemble simulations for a production workflow using the MLDA algorithm for Bayesian inference of a parameter that governs the dynamic rupture process. With fused simulations, the amount of node hours consumed is reduced by 18% for this workflow.

The observed performance improvements, but also their variability w.r.t. setup, motivate more detailed studies on the performance of the various kernels on various platforms, which is beyond the scope of this paper. Further research questions include studying the impact of local time stepping in more detail, and also examining fused ensemble simulations on GPU architectures. If the role of the SIMD length is taken by the warp size (typically 32) on GPU architectures, for example, it is questionable whether fused ensembles over 32 simulations can be feasible.

Probably the most promising feature of fused simulations is that they work best for small, low-order simulations, which form the bulk of simulations in multi-level UQ algorithms. We therefore plan to experiment with larger workflows and more complicated multi-level structures for MLDA, but also to investigate other UQ algorithms than MLDA. We also see potential for further improvement in the interaction of MLDA/UQ algorithm, UM-Bridge, and SeisSol to efficiently schedule fused ensemble simulations for Bayesian inversion.

10 Declarations

Funding

This work has been funded by the Innovation Study ScalaMIDA, which has received funding through the Inno4scale project, which is funded by the European High Performance Computing Joint Undertaking (JU) under Grant Agreement No. 101118139. The EuroHPC JU receives support from the European Union's Horizon Europe Programme. We also acknowledge funding for the ChEESE-2P cluster of excellence (also as part of EuroHPC JU) by Horizon Europe (grant agreement No. 101093038) and by the German Federal Ministry of Research, Technology and Space. Z.N. and A.-A.G. acknowledge additional support from the European Union's Horizon 2020 research and innovation programme under the Marie-Sklodowska-Curie grant agreement No. 955515 – SPIN ITN (www.spin-itn.eu) and Horizon Europe (DT-GEO, grant no. 101058129; Geo-INQUIRE, grant no. 101058518), the National Science Foundation (grant nos. EAR-2225286, EAR-2121568, OAC-2311208 and OAC-2311206), the National Aeronautics and Space Administration (grant no. 80NSSC20K0495) and the Statewide California Earthquake Center (SCEC projects 25341, 25259). Compute resources have been provided by the Texas Advanced Computing Center (TACC) via project EAR20006 (Frontera), and via Leadership Resource Allocation (LRAC) project EAR25002 (Vista). For compute resources on SuperMUC-NG at Leibniz Supercomputing Centre (www.lrz.de), the authors gratefully acknowledge the Gauss Centre for Supercomputing e.V. (www.gauss-centre.eu, project pn52yu).

Acknowledgements

We especially thank Sebastian Wolf and Maximilian Kruse for their previous work (Kruse et al. 2024) on the MLDA workflow with SeisSol. In addition, we thank all colleagues, especially Ian Wang, at the Texas Advanced Computing Center (TACC, http://www.tacc.utexas.edu) at The University of Texas at Austin for their excellent support regarding Frontera and Vista.

Competing interests

All authors certify that they have no affiliations with or involvement in any organization or entity with any financial interest or non-financial interest in the subject matter or materials discussed in this manuscript

Code availability

SeisSol (www.seissol.org) and UM-Bridge are available as open source software. The code versions of SeisSol used in this paper for fused and serial simulations are available from https://github.com/SeisSol/SeisSol/tree/UQ-fused and https://github.com/SeisSol/SeisSol/tree/UQ-serial, respectively. UM-Bridge is available from https://github.com/UM-Bridge/umbridge, the batcher script can be found at https://github.com/SeisSolUQ/Batcher, and the SeisSol UMBridge server is published at https://github.com/SeisSolUQ/Seis-Bridge/tree/main/ridgecrest.

Author contribution information

Conceptualisation: V. Kurapati, M. Bader, A.-A. Gabriel, L. Seelinger; Methodology: L. Seelinger, V. Kurapati; Performance Optimization: David Schneller, V. Kurapati; Formal analysis and investigation: V. Kurapati, D. Schneller, L. Seelinger, Z. Niu, M. Bader; Writing – original draft preparation: V. Kurapati, David Schneller, L. Seelinger, A.-A. Gabriel, M. Bader; Writing – review and editing: V. Kurapati, David Schneller, L. Seelinger, A.-A. Gabriel, M. Bader; Funding acquisition: M. Bader, A.-A. Gabriel, L. Seelinger; Resources: M. Bader, A.-A. Gabriel; Supervision: M. Bader, A.-A. Gabriel.

References

Akcelik, V., Bielak, J., Biros, G., Epanomeritakis, I., Fernandez, A., Ghattas, O., Kim, E.J., Lopez, J., O'Hallaron, D., Tu, T., Urbanic, J.: High resolution forward and inverse earthquake modeling on terascale computers. In: Proceedings of the 2003 ACM/IEEE Conference on Supercomputing. SC '03, p. 52. Association for Computing Machinery, New York, NY, USA (2003). https://doi.org/10.1145/1048935.1050202

Abdelfattah, A., Haidar, A., Tomov, S., Dongarra, J.: Performance, design, and autotuning of batched GEMM for GPUs. In: Kunkel, J.M., Balaji, P., Dongarra, J. (eds.) High Performance Computing, pp. 21–38. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-41321-1_2

Andrews, D.J.: Rupture dynamics with energy loss outside the slip zone. Journal of Geophysical Research: Solid Earth **110**(B1) (2005) https://doi.org/10.1029/2004JB003191

Bastian, P., Altenbernd, M., Dreier, N.-A., Engwer, C., Fahlke, J., Fritze, R., Geveler, M., Göddeke, D., Iliev, O., Ippisch, O., Mohring, J., Müthing, S., Ohlberger, M., Ribbrock, D., Shegunov, N., Turek, S.: Exa-Dune—flexible PDE solvers, numerical

methods and applications. In: Bungartz, H.-J., Reiz, S., Uekermann, B., Neumann, P., Nagel, W.E. (eds.) Software for Exascale Computing – SPPEXA 2016-2019, pp. 225–269. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-47956-5_9

Breuer, A., Heinecke, A., Bader, M.: Petascale local time stepping for the ADER-DG finite element method. In: 2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 854–863 (2016). https://doi.org/10.1109/IPDPS.2016.109

Breuer, A., Heinecke, A., Cui, Y.: EDGE: Extreme scale fused seismic simulations with the discontinuous Galerkin method. In: Kunkel, J.M., Yokota, R., Balaji, P., Keyes, D. (eds.) High Performance Computing, pp. 41–60. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-58667-0_3

Christen, J.A., Fox, C.: Markov chain Monte Carlo using an approximation. Journal of Computational and Graphical Statistics **14**(4), 795–810 (2005) https://doi.org/10.1198/106186005X76983

Cockburn, B., Karniadakis, G.E., Shu, C.-W.: The development of discontinuous Galerkin methods. In: Cockburn, B., Karniadakis, G.E., Shu, C.-W. (eds.) Discontinuous Galerkin Methods, pp. 3–50. Springer, Berlin, Heidelberg (2000). https://doi.org/10.1007/978-3-642-59721-3_1

Callaghan, S., Maechling, P.J., Silva, F., Su, M.-H., Milner, K.R., Graves, R.W., Olsen, K.B., Cui, Y., Vahi, K., Kottke, A., Goulet, C.A., Deelman, E., Jordan, T.H., Ben-Zion, Y.: Using open-science workflow tools to produce SCEC CyberShake physics-based probabilistic seismic hazard models. Frontiers in High Performance Computing **2**, 1–21 (2024) https://doi.org/10.3389/fhpcp.2024.1360720

Cui, Y., Olsen, K.B., Jordan, T.H., Lee, K., Zhou, J., Small, P., Roten, D., Ely, G., Panda, D.K., Chourasia, A., Levesque, J., Day, S.M., Maechling, P.: Scalable earthquake simulation on petascale supercomputers. In: SC '10: Proceedings of the 2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–20 (2010). https://doi.org/10.1109/SC.2010.45

Cui, Y., Poyraz, E., Olsen, K.B., Zhou, J., Withers, K., Callaghan, S., Larkin, J., Guest, C., Choi, D., Chourasia, A., Shi, Z., Day, S.M., Maechling, P.J., Jordan, T.H.: Physics-based seismic hazard analysis on petascale heterogeneous supercomputers. In: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis. SC '13. Association for Computing Machinery, New York, NY, USA (2013). https://doi.org/10.1145/2503210.2503300

Carter Edwards, H., Trott, C.R., Sunderland, D.: Kokkos: Enabling manycore performance portability through polymorphic memory access patterns. Journal of Parallel and Distributed Computing **74**(12), 3202–3216 (2014) https://doi.org/10.1016/j.jpdc.2014.07.003 . Domain-Specific Languages and High-Level Frameworks

for High-Performance Computing

de la Puente, J., Ampuero, J.-P., Käser, M.: Dynamic rupture modeling on unstructured meshes using a discontinuous Galerkin method. Journal of Geophysical Research: Solid Earth **114**(B10) (2009) https://doi.org/10.1029/2008JB006271

Dunham, E.M., Belanger, D., Cong, L., Kozdon, J.E.: Earthquake ruptures with strongly rate-weakening friction and off-fault plasticity, part 1: Planar faults. Bulletin of the Seismological Society of America **101**(5), 2296–2307 (2011) https://doi.org/10.1785/0120100075

Dumbser, M., Balsara, D.S., Toro, E.F., Munz, C.-D.: A unified framework for the construction of one-step finite volume and discontinuous Galerkin schemes on unstructured meshes. Journal of Computational Physics **227**(18), 8209–8253 (2008) https://doi.org/10.1016/j.jcp.2008.05.025

Dongarra, J., Hammarling, S., Higham, N.J., Relton, S.D., Valero-Lara, P., Zounon, M.: The design and performance of batched BLAS on modern high-performance computing systems. Procedia Computer Science **108**, 495–504 (2017) https://doi.org/10.1016/j.procs.2017.05.138 . International Conference on Computational Science, ICCS 2017, 12-14 June 2017, Zurich, Switzerland

Dieterich, J.H.: Modeling of rock friction: 1. experimental results and constitutive equations. Journal of Geophysical Research: Solid Earth **84**(B5), 2161–2168 (1979) https://doi.org/10.1029/JB084iB05p02161

Dumbser, M., Käser, M.: An arbitrary high-order discontinuous Galerkin method for elastic waves on unstructured meshes — ii. the three-dimensional isotropic case. Geophysical Journal International **167**(1), 319–336 (2006) https://doi.org/10.1111/j.1365-246X.2006.03120.x

Dodwell, T.J., Ketelsen, C., Scheichl, R., Teckentrup, A.L.: A hierarchical multilevel Markov Chain Monte Carlo algorithm with applications to uncertainty quantification in subsurface flow. SIAM/ASA Journal on Uncertainty Quantification **3**(1), 1075–1108 (2015) https://doi.org/10.1137/130915005

Dodwell, T.J., Ketelsen, C., Scheichl, R., Teckentrup, A.L.: Multilevel Markov Chain Monte Carlo. SIAM Review **61**(3), 509–545 (2019) https://doi.org/10.1137/19M126966X

Dumbser, M., Käser, M., Toro, E.F.: An arbitrary high-order Discontinuous Galerkin method for elastic waves on unstructured meshes – V. Local time stepping and p-adaptivity. Geophysical Journal International **171**(2), 695–717 (2007) https://doi.org/10.1111/j.1365-246X.2007.03427.x

Erickson, B.A., Dunham, E.M., Khosravifar, A.: A finite difference method for off-fault plasticity throughout the earthquake cycle. Journal of the Mechanics and Physics

of Solids **109**, 50–77 (2017) https://doi.org/10.1016/j.jmps.2017.08.002

Floyd, M., Funning, G., Fialko, Y., Terry, R., Herring, T.: Survey and continuous GNSS in the vicinity of the july 2019 Ridgecrest earthquakes. Seismological Research Letters **91**(4), 2047–2054 (2020) https://doi.org/10.1785/0220190324

Feng, Y.T., Owen, D.R.J., Perić, D.: A block conjugate gradient method applied to linear systems with multiple right-hand sides. Computer Methods in Applied Mechanics and Engineering **127**(1), 203–215 (1995) https://doi.org/10.1016/0045-7825(95)00832-2

Gunzburger, M., Jiang, N., Wang, Z.: An efficient algorithm for simulating ensembles of parameterized flow problems. IMA Journal of Numerical Analysis **39**(3), 1180–1205 (2018) https://doi.org/10.1093/imanum/dry029

Gabriel, A., Ulrich, T., Marchandon, M., Biemiller, J., Rekoske, J.: 3d dynamic rupture modeling of the 6 february 2023, Kahramanmaras, Turkey Mw 7.8 and 7.7 earthquake doublet using early observations. The Seismic Record **3**(4), 342–356 (2023) https://doi.org/10.1785/0320230028

Harris, R.A., Barall, M., Aagaard, B., Ma, S., Roten, D., Olsen, K., Duan, B., Liu, D., Luo, B., Bai, K., Ampuero, J., Kaneko, Y., Gabriel, A., Duru, K., Ulrich, T., Wollherr, S., Shi, Z., Dunham, E., Bydlon, S., Zhang, Z., Chen, X., Somala, S.N., Pelties, C., Tago, J., Cruz-Atienza, V.M., Kozdon, J., Daub, E., Aslam, K., Kase, Y., Withers, K., Dalguer, L.: A suite of exercises for verifying dynamic earthquake rupture codes. Seismological Research Letters **89**(3), 1146–1162 (2018) https://doi.org/10.1785/0220170222

Heroux, M.A., Bartlett, R.A., Howle, V.E., Hoekstra, R.J., Hu, J.J., Kolda, T.G., Lehoucq, R.B., Long, K.R., Pawlowski, R.P., Phipps, E.T., Salinger, A.G., Thornquist, H.K., Tuminaro, R.S., Willenbring, J.M., Williams, A., Stanley, K.S.: An overview of the Trilinos project. ACM Trans. Math. Softw. **31**(3), 397–423 (2005) https://doi.org/10.1145/1089014.1089021

Heinecke, A., Breuer, A., Rettenberger, S., Bader, M., Gabriel, A.-A., Pelties, C., Bode, A., Barth, W., Liao, X.-K., Vaidyanathan, K., Smelyanskiy, M., Dubey, P.: Petascale high order dynamic rupture earthquake simulations on heterogeneous supercomputers. In: SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 3–14 (2014). https://doi.org/10.1109/SC.2014.6

Heinecke, A., Henry, G., Hutschinson, M., Pabst, H.: LIBXSMM: accelerating small matrix multiplications by runtime code generation. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–11. IEEE Press, Salt Lake City, Utah (2016). https://doi.org/10.5555/3014904.3015017

Ida, Y.: Cohesive force across the tip of a longitudinal-shear crack and Griffith's specific surface energy. Journal of Geophysical Research (1896-1977) **77**(20), 3796–3805 (1972) https://doi.org/10.1029/JB077i020p03796

Imamura, S., Ono, K., Yokokawa, M.: Iterative-method performance evaluation for multiple vectors associated with a large-scale sparse matrix. International Journal of Computational Fluid Dynamics **30**(6), 395–401 (2016) https://doi.org/10.1080/10618562.2016.1234046

Jia, Z., Jin, Z., Marchandon, M., Ulrich, T., Gabriel, A.-A., Fan, W., Shearer, P., Zou, X., Rekoske, J., Bulut, F., Garagon, A., Fialko, Y.: The complex dynamics of the 2023 Kahramanmaras, Turkey, Mw 7.8–7.7 earthquake doublet. Science **381**(6661), 985–990 (2023) https://doi.org/10.1126/science.adi0685

Käser, M., Dumbser, M.: An arbitrary high-order discontinuous Galerkin method for elastic waves on unstructured meshes — i. the two-dimensional isotropic case with external source terms. Geophysical Journal International **166**(2), 855–877 (2006) https://doi.org/10.1111/j.1365-246X.2006.03051.x

Käser, M., Dumbser, M., de la Puente, J., Igel, H.: An arbitrary high-order discontinuous Galerkin method for elastic waves on unstructured meshes — iii. viscoelastic attenuation. Geophysical Journal International **168**(1), 224–242 (2007) https://doi.org/10.1111/j.1365-246X.2006.03193.x

Kaneko, Y., Fialko, Y.: Shallow slip deficit due to large strike-slip earthquakes in dynamic rupture simulations with elasto-plastic off-fault response. Geophysical Journal International **186**(3), 1389–1403 (2011) https://doi.org/10.1111/j.1365-246X.2011.05117.x

Kühn, M.J., Holke, J., Lutz, A., Thies, J., Röhrig-Zöllner, M., Bleh, A., Backhaus, J., Basermann, A.: SIMD vectorization for simultaneous solution of locally varying linear systems with multiple right-hand sides. The Journal of Supercomputing **79**(13), 14684–14706 (2023) https://doi.org/10.1007/s11227-023-05220-4

Kruse, M., Niu, Z., Wolf, S., Lykkegaard, M., Bader, M., Gabriel, A.-A., Seelinger, L.: Scalable Bayesian inference of large simulations via asynchronous prefetching Multilevel Delayed Acceptance. In: Proceedings of the Platform for Advanced Scientific Computing Conference. PASC '25, pp. 1–13. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3732775.3733581

Krasnopolsky, B.I.: An approach for accelerating incompressible turbulent flow simulations based on simultaneous modelling of multiple ensembles. Computer Physics Communications **229**, 8–19 (2018) https://doi.org/10.1016/j.cpc.2018.03.023

Krenz, L., Uphoff, C., Ulrich, T., Gabriel, A.-A., Abrahams, L.S., Dunham, E.M., Bader, M.: 3d acoustic-elastic coupling with gravity: the dynamics of the 2018

Palu, Sulawesi earthquake and tsunami. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. SC '21. Association for Computing Machinery, New York, NY, USA (2021). https://doi.org/10.1145/3458817.3476173

Liu, X., Chow, E., Vaidyanathan, K., Smelyanskiy, M.: Improving the performance of dynamical simulations via multiple right-hand sides. In: 2012 IEEE 26th International Parallel and Distributed Processing Symposium, pp. 36–47 (2012). https://doi.org/10.1109/IPDPS.2012.14

Lykkegaard, M.B., Dodwell, T.J., Fox, C., Mingas, G., Scheichl, R.: Multilevel delayed acceptance MCMC. SIAM/ASA Journal on Uncertainty Quantification **11**(1), 1–30 (2023) https://doi.org/10.1137/22M1476770

Masliah, I., Abdelfattah, A., Haidar, A., Tomov, S., Baboulin, M., Falcou, J., Dongarra, J.: High-performance matrix-matrix multiplications of very small matrices. In: Dutot, P.-F., Trystram, D. (eds.) Euro-Par 2016: Parallel Processing, pp. 659–671. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-43659-3_48

Mert, A., Fahjan, Y.M., Hutchings, L.J., Pınar, A.: Physically based probabilistic seismic hazard analysis using broadband ground motion simulation: a case study for the Prince Islands fault, Marmara sea. Earth, Planets and Space **68** (2016) https://doi.org/10.1186/s40623-016-0520-3

Melgar, D., Melbourne, T.I., Crowell, B.W., Geng, J., Szeliga, W., Scrivner, C., Santillan, M., Goldberg, D.E.: Real-time high-rate GNSS displacements: Performance demonstration during the 2019 Ridgecrest, California, earthquakes. Seismological Research Letters **91**(4), 1943–1951 (2020) https://doi.org/10.1785/0220190223

Makarashvili, V., Merzari, E., Obabko, A., Siegel, A., Fischer, P.: A performance analysis of ensemble averaging for high fidelity turbulence simulations at the strong scaling limit. Computer Physics Communications **219**, 236–245 (2017) https://doi.org/10.1016/j.cpc.2017.05.023

Mohebujjaman, M., Rebholz, L.G.: An efficient algorithm for computation of MHD flow ensembles. Computational Methods in Applied Mathematics **17**(1), 121–137 (2017) https://doi.org/10.1515/cmam-2016-0033

Niu, Z., Gabriel, A.-A., Wolf, S., Ulrich, T., Lyakhovsky, V., Igel, H.: A discontinuous Galerkin method for simulating 3D seismic wave propagation in nonlinear rock models: Verification and application to the mw 7.8 2015 Gorkha, Nepal Earthquake. Journal of Geophysical Research: Solid Earth **130**(7), 2025–031378 (2025) https://doi.org/10.1029/2025JB031378 https://agupubs.onlinelibrary.wiley.com/doi/pdf/10.1029/2025JB031378. e2025JB031378 2025JB031378

Niu, Z.: Regional-scale Forward Modeling and Inversion of 3D Wave Propagation and

Dynamic Rupture Processes with Nonlinear Mechanical Models of Rocks and Soils. http://nbn-resolving.de/urn:nbn:de:bvb:19-354384

Pelties, C., de la Puente, J., Ampuero, J.-P., Brietzke, G.B., Käser, M.: Three-dimensional dynamic rupture simulation with a high-order discontinuous Galerkin method on unstructured tetrahedral meshes. Journal of Geophysical Research: Solid Earth **117**(B2) (2012) https://doi.org/10.1029/2011JB008857

Phipps, E., D'Elia, M., Edwards, H.C., Hoemmen, M., Hu, J., Rajamanickam, S.: Embedded ensemble propagation for improving performance, portability, and scalability of uncertainty quantification on emerging computational architectures. SIAM Journal on Scientific Computing **39**(2), 162–193 (2017) https://doi.org/10.1137/15M1044679

Parno, M., Davis, A., Seelinger, L.: MUQ: the MIT Uncertainty Quantification Library. Journal of Open Source Software **6**(68), 3076 (2021) https://doi.org/10.21105/joss.03076

Premus, J., Gallovič, F., Ampuero, J.-P.: Bridging time scales of faulting: From coseismic to postseismic slip of the Mw 6.0 2014 South Napa, California earthquake. Science advances **8**(38), 2536 (2022) https://doi.org/10.1126/sciadv.abq2536

Palmer, A.C., Rice, J.R., Hill, R.: The growth of slip surfaces in the progressive failure of over-consolidated clay. Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences **332**(1591), 527–548 (1973) https://doi.org/10.1098/rspa.1973.0040

Petersson, N.A., Sjogreen, B., Tang, H., Pankajakshan, R.: SW4, Version 3.0. https://doi.org/10.5281/zenodo.8322590

Richtmann, D., Heybrock, S., Wettig, T.: Multiple right-hand-side setup for the DD-αAMG. In: Proceedings of The 33rd International Symposium on Lattice Field Theory — PoS(LATTICE 2015), vol. 251, p. 035 (2016). https://doi.org/10.22323/1.251.0035

Rojas, O., Monterrubio-Velasco, M., Rodríguez, J.E., Callaghan, S., Abril, C., Halldorsson, B., Kowsari, M., Bayat, F., Olsen, K.B., Gabriel, A., de la Puente, J.: Earthquake fault rupture modeling and ground-motion simulations for the southwest Iceland transform zone using CyberShake. Bulletin of the Seismological Society of America **115**(1), 69–85 (2024) https://doi.org/10.1785/0120240064

Ruina, A.: Slip instability and state variable friction laws. Journal of Geophysical Research: Solid Earth **88**(B12), 10359–10370 (1983) https://doi.org/10.1029/JB088iB12p10359

Schliwa, N., Gabriel, A.-A., Premus, J., Gallovič, F.: The linked complexity of coseismic and postseismic faulting revealed by seismo-geodetic dynamic inversion of the

2004 Parkfield earthquake. Journal of Geophysical Research: Solid Earth **129**(12), 2024–029410 (2024) https://doi.org/10.1029/2024JB029410

Seelinger, L., Reinarz, A., Lykkegaard, M.B., Akers, R., Alghamdi, A.M.A., Aristoff, D., Bangerth, W., Bénézech, J., Diez, M., Frey, K., Jakeman, J.D., Jørgensen, J.S., Kim, K.-T., Kent, B.M., Martinelli, M., Parno, M., Pellegrini, R., Petra, N., Riis, N.A.B., Rosenfeld, K., Serani, A., Tamellini, L., Villa, U., Dodwell, T.J., Scheichl, R.: Democratizing uncertainty quantification. Journal of Computational Physics **521**, 113542 (2025) https://doi.org/10.1016/j.jcp.2024.113542

Taufiqurrahman, T., Gabriel, A.-A., Li, D., Ulrich, T., Li, B., Carena, S., Verdecchia, A., Gallovič, F.: Dynamics, interactions and delays of the 2019 Ridgecrest rupture sequence. Nature **618**(7964), 308–315 (2023) https://doi.org/10.1038/s41586-023-05985-x

Taufiqurrahman, T., Gabriel, A.-A., Ulrich, T., Valentová, L., Gallovič, F.: Broadband dynamic rupture modeling with fractal fault roughness, frictional heterogeneity, viscoelasticity and topography: The 2016 Mw 6.2 Amatrice, Italy Earthquake. Geophysical Research Letters **49**(22), 2022–098872 (2022) https://doi.org/10.1029/2022GL098872 https://agupubs.onlinelibrary.wiley.com/doi/pdf/10.1029/2022GL098872. e2022GL098872 2022GL098872

Templeton, E.L., Rice, J.R.: Off-fault plasticity and earthquake rupture dynamics: 1. dry materials or neglect of fluid pressure changes. Journal of Geophysical Research: Solid Earth **113**(B9) (2008) https://doi.org/10.1029/2007JB005529

Titarev, V.A., Toro, E.F.: ADER: Arbitrary high order Godunov approach. J. Sci. Comput. **17**(1–4), 609–618 (2002) https://doi.org/10.1023/A:1015126814947

Uphoff, C., Bader, M.: Generating high performance matrix kernels for earthquake simulations with viscoelastic attenuation. In: 2016 International Conference on High Performance Computing & Simulation (HPCS), pp. 908–916 (2016). https://doi.org/10.1109/HPCSim.2016.7568431

Uphoff, C., Bader, M.: Yet another tensor toolbox for discontinuous Galerkin methods and other applications. ACM Trans. Math. Softw. **46**(4) (2020) https://doi.org/10.1145/3406835

Uphoff, C.: Flexible model extension and optimisation for earthquake simulations at extreme scales. PhD thesis, Technische Universität München (2020). https://mediatum.ub.tum.de/1531433

Uphoff, C., Rettenberger, S., Bader, M., Madden, E.H., Ulrich, T., Wollherr, S., Gabriel, A.-A.: Extreme scale multi-physics simulations of the tsunamigenic 2004 Sumatra Megathrust Earthquake. In: SC '17: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pp.

21–12116. ACM, New York, NY, USA (2017). https://doi.org/10.1145/3126908.3126948

Wollherr, S., Gabriel, A.-A., Uphoff, C.: Off-fault plasticity in three-dimensional dynamic rupture simulations using a modal discontinuous Galerkin method on unstructured meshes: implementation, verification and application. Geophysical Journal International **214**(3), 1556–1584 (2018) https://doi.org/10.1093/gji/ggy213

Wolf, S., Galis, M., Uphoff, C., Gabriel, A.-A., Moczo, P., Gregor, D., Bader, M.: An efficient ADER-DG local time stepping scheme for 3d HPC simulation of seismic waves in poroelastic media. Journal of Computational Physics **455**, 110886 (2022) https://doi.org/10.1016/j.jcp.2021.110886

Walden, A., Khan, S., Joó, B., Ranjan, D., Zubair, M.: Optimizing a multiple right-hand side Dslash kernel for Intel Knights Corner. In: Taufer, M., Mohr, B., Kunkel, J.M. (eds.) High Performance Computing, pp. 390–401. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-46079-6_28

Wolf, S.F.: High-performance earthquake simulations with advanced material models. PhD thesis, Technische Universität München (2024). https://doi.org/10.14459/2024md1729905

Xu, S., Ben-Zion, Y., Ampuero, J.-P.: Properties of inelastic yielding zones generated by in-plane dynamic ruptures—i. model description and basic results. Geophysical Journal International **191**(3), 1325–1342 (2012) https://doi.org/10.1111/j.1365-246X.2012.05679.x

Yang, W., Fang, J., Dong, D., Su, X., Wang, Z.: LIBSHALOM: optimizing small and irregular-shaped matrix multiplications on ARMv8 multi-cores. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. SC '21. Association for Computing Machinery, New York, NY, USA (2021). https://doi.org/10.1145/3458817.3476217

Yao, J., Shi, B., Xiang, C., Jia, H., Li, C., Cao, H., Zhang, Y.: IAAT: a input-aware adaptive tuning framework for small GEMM. In: 2021 IEEE 27th International Conference on Parallel and Distributed Systems (ICPADS), pp. 899–906. IEEE Computer Society, Los Alamitos, CA, USA (2021). https://doi.org/10.1109/ICPADS53394.2021.00118

Zuccolo, E., Bolzon, G., Pitari, F., Muñoz, L., Scaini, C., Vanini, M., Poggi, V., Salon, S.: Advancing rapid response to earthquakes with tiered physics-based ground-shaking simulations: The UrgentShake system. Seismological Research Letters (2025) https://doi.org/10.1785/0220240472